THE HANDWRITTEN INTERVIEW NOTEBOOK SERIES





GIT

Zero to Hero
Premium Field Notes

Short answers. Real commands. Clear commit diagrams. Practical mistakes explained simply.

110 ORIGINAL QUESTIONS + 110 answered follow-ups
22 TOPIC CHAPTERS + 14 verified command labs
BEGINNER TO SENIOR • practice with confidence

Java delivery workflows • Rebase & recovery • Safe team collaboration
Prepared from the supplied Git interview guide • notebook edition 2026

How to use these notes

Every original interview question is reproduced in the same order and wording as the source PDF. The answer is kept short and precise; the practical command example, real behavior and danger signs help you understand why it works.

① **Say the answer** in your own words (30 seconds).

② **Read the command** and predict whether it changes HEAD, index, files or a remote reference.

③ **Answer the follow-up** before reading its purple answer box.

④ **Sketch the concept** on paper and explain one failure.

⑤ **Run the labs** only in separate empty practice folders.

The mini concept maps are learning aids, not exhaustive Git object graphs. Sample commands may use placeholders such as `BAD_COMMIT`; replace them with the correct IDs, and do not run alternatives in sequence unless the exercise explicitly says to do so.

Safety: Hard reset, clean, force-push and history rewrite can permanently affect work or collaborators. Always inspect status, create backups where appropriate, and follow your team's branch protections.

Your learning map

22 chapters • 110 questions • 110 follow-ups with direct answers

01 Git fundamentals and the right mental model (Q001–Q005)

02 Working tree, index and staging (Q006–Q010)

03 Setup, configuration and cloning (Q011–Q015)

04 Commits, inspection and useful history queries (Q016–Q020)

05 Branches, switching and detached HEAD (Q021–Q025)

06 Merging and resolving conflicts (Q026–Q030)

07 Rebase and rewriting a private branch (Q031–Q035)

08 Cherry-pick, amend and interactive cleanup (Q036–Q040)

09 Restore, reset and revert without confusion (Q041–Q045)

10 Reflog, recovery and clean-up (Q046–Q050)

11 Remotes, fetch and pull (Q051–Q055)

12 Push, rejected updates and leases (Q056–Q060)

13 Tags, releases and reproducible builds (Q061–Q065)

14 Stash and multiple worktrees (Q066–Q070)

15 Ignore rules, attributes and cross-platform files (Q071–Q075)

16 Git objects, references and the index (Q076–Q080)

17 Debugging with bisect, blame and history search (Q081–Q085)

18 Large repositories, sparse checkout, submodules and LFS (Q086–Q090)

19 Team workflows, review and Java CI (Q091–Q095)

20 Hooks, credentials and secure history (Q096–Q100)

21 Advanced revisions, patches and repeated resolutions (Q101–Q105)

22 Senior interview scenarios and judgment (Q106–Q110)

Then move to fourteen Bash labs, a quick Git command chooser, real incident examples and official references.

CHAPTER 01 / Git fundamentals and the right mental model Q 001 of 110

INTERVIEW QUESTION / Q001

What is Git and why is it called distributed?

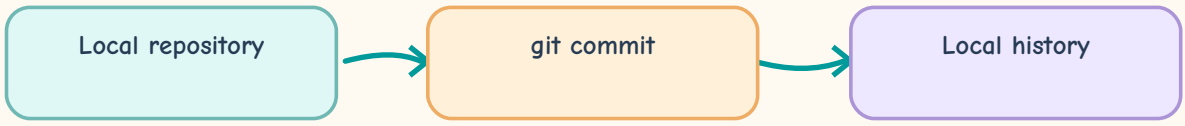
THE SHORT ANSWER

Git is a version-control system. Each normal full clone has local history and can create commits without contacting a central server; sharing happens through explicit network operations.

COMMAND EXAMPLE / SCENARIO

```
git log --oneline
   git add src/OrderService.java
   git commit -m "Validate positive order quantity"
```

DRAW IT TO REMEMBER



Most Git work is offline; sharing uses push.

WHAT REALLY HAPPENS

The commit is local. A later push publishes reachable objects and updates a remote reference if permitted. Shallow and partial clones intentionally omit some history or objects, so do not assume every clone contains everything.

FOLLOW-UP QUESTION + ANSWER

Which Git operations can you perform without internet access?

You can edit files, stage, commit, branch, switch, inspect history, diff, merge and rebase using objects already available locally. Fetch and push normally need a reachable remote.

⚠ **INTERVIEW TRAP:** Committing does not automatically publish your work to teammates.

Original question & example: uploaded Git guide, source page 5. Follow-up answer: added for this edition.

CHAPTER 01 / Git fundamentals and the right mental model Q 002 of 110

INTERVIEW QUESTION / Q002

What is the difference between Git and GitHub?

THE SHORT ANSWER

Git is the version-control tool and data model. GitHub is one hosting and collaboration platform that can store Git repositories and provide reviews, permissions and automation.

COMMAND EXAMPLE / SCENARIO

```
Git: commit, branch, merge, fetch.
   Hosting platform: pull request, review, protected branch.
   Remote name: origin; it may point to any supported server.
```

✎ REMEMBER Git: commit, branch, merge, fetch.

WHAT REALLY HAPPENS

A repository can be hosted locally or by another provider. A pull request is a collaboration request defined by the platform, not a Git object type. Server policy can reject a push even when the local history is valid.

FOLLOW-UP QUESTION + ANSWER

Can a Git repository exist without any hosting account?

Yes. `git init` creates a local repository. GitHub, GitLab or another server is optional for hosting and collaboration.

⚠ INTERVIEW TRAP: Git pull and a pull request are different concepts.

Original question & example: uploaded Git guide, source page 5. Follow-up answer: added for this edition.

CHAPTER 01 / Git fundamentals and the right mental model Q 003 of 110

INTERVIEW QUESTION / Q003

Does Git store changes or snapshots?

THE SHORT ANSWER

The logical model is a snapshot for each commit, with unchanged content reused by identity. Storage may use compression and deltas internally, but those are implementation details.

COMMAND EXAMPLE / SCENARIO

```
Commit A: Order.java version 1; Customer.java version 1.
   Commit B: Order.java version 2; Customer.java version 1.
   The unchanged Customer content can be shared.
```

✎ **REMEMBER** Commit A: Order.java version 1; Customer.java version 1.

WHAT REALLY HAPPENS

A commit points to a root tree describing tracked paths. Diff computes a comparison between snapshots. This explains why Git can inspect an old state without replaying a sequence of edits from the beginning.

FOLLOW-UP QUESTION + ANSWER

What object connects a commit to its file hierarchy?

A tree object. Each commit names a root tree, which links directory entries to blobs and nested trees.

⚠ **INTERVIEW TRAP:** A commit is not simply a text patch file stored by Git.

Original question & example: uploaded Git guide, source page 6. Follow-up answer: added for this edition.

CHAPTER 01 / Git fundamentals and the right mental model Q 004 of 110

INTERVIEW QUESTION / Q004

What is a commit and what does its identifier mean?

THE SHORT ANSWER

A commit is an immutable object containing a tree, parent references and metadata. Its object ID is derived from its encoded contents, so changing those contents changes the ID.

COMMAND EXAMPLE / SCENARIO

```
git show --no-patch --format=fuller HEAD
   git rev-parse HEAD
   git cat-file -p HEAD
```

✎ REMEMBER git show --no-patch --format=fuller HEAD

WHAT REALLY HAPPENS

The parent relationship is part of the object. Rebase or amend can create a new ID even when a file snapshot stays the same. ID length depends on repository object format, and abbreviated IDs must be unambiguous within the repository.

FOLLOW-UP QUESTION + ANSWER

Why does changing only a commit message create a new commit ID?

The message is part of the commit object's contents. Changing it changes the object's hash, even when the file tree is identical.

⚠ **INTERVIEW TRAP:** A short hash is an abbreviation, not a sequential commit number.

Original question & example: uploaded Git guide, source page 6. Follow-up answer: added for this edition.

CHAPTER 01 / Git fundamentals and the right mental model Q 005 of 110

INTERVIEW QUESTION / Q005

What are HEAD, a branch and a tag?

THE SHORT ANSWER

HEAD identifies the checked-out position, usually through a branch reference. A branch names a movable tip. A tag names a chosen object, commonly a release commit.

COMMAND EXAMPLE / SCENARIO

```
git symbolic-ref --short HEAD
   git branch --show-current
   git tag v1.0.0
```

✎ REMEMBER git symbolic-ref --short HEAD

WHAT REALLY HAPPENS

On a branch, committing creates a child of the current tip and moves that branch to the new commit. In detached HEAD, HEAD points directly to a commit instead. Tags do not advance with normal commits.

FOLLOW-UP QUESTION + ANSWER

What reference moves when you commit on feature/orders?

`refs/heads/feature/orders` moves to the new commit. `main` stays where it was.

⚠ INTERVIEW TRAP: A branch is a reference, not a copied directory of files.

Original question & example: uploaded Git guide, source page 6. Follow-up answer: added for this edition.

CHAPTER 02 / Working tree, index and staging Q 006 of 110

INTERVIEW QUESTION / Q006

What are the working tree, index and repository?

THE SHORT ANSWER

The working tree contains checked-out files. The index records the proposed next commit snapshot. The repository stores objects and references representing history.

COMMAND EXAMPLE / SCENARIO

```
Edit file -> working tree changes.
   git add file -> copy its current content into the index.
   git commit -> create a commit from the index.
```

DRAW IT TO REMEMBER



Commit reads the index, not unstaged edits.

WHAT REALLY HAPPENS

Staging does not freeze the working file. If you edit it again, the staged version and working version differ. This lets one file have both staged and unstaged changes at the same time.

FOLLOW-UP QUESTION + ANSWER

Which version is committed if you edit again after git add?

The version captured by the last `git add` is committed. The edit made afterward remains unstaged in your working file.

⚠ **INTERVIEW TRAP:** A normal commit includes staged content, not every current file edit.

Original question & example: uploaded Git guide, source page 7. Follow-up answer: added for this edition.

CHAPTER 02 / Working tree, index and staging Q 007 of 110

INTERVIEW QUESTION / Q007

How do you read git status --short?

THE SHORT ANSWER

The first status column compares the index with HEAD; the second compares the working tree with the index. Untracked paths appear with question marks.

COMMAND EXAMPLE / SCENARIO

```
git status --short
   M  src/Order.java   # staged modification
    M src/Customer.java  # unstaged modification
   MM src/Price.java   # staged, then edited again
```

✎ REMEMBER git status --short

WHAT REALLY HAPPENS

Unmerged statuses describe conflict stages and need separate attention. The long status view gives operation guidance. Read status before applying an undo command because the correct action depends on which area contains the work.

FOLLOW-UP QUESTION + ANSWER

How would you inspect the two changes represented by MM?

Run `git diff --staged -- FILE` for HEAD-to-index changes and `git diff -- FILE` for index-to-working-file changes.

⚠ **INTERVIEW TRAP:** Treating both M columns as identical can cause you to discard the wrong version.

Original question & example: uploaded Git guide, source page 7. Follow-up answer: added for this edition.

CHAPTER 02 / Working tree, index and staging Q 008 of 110

INTERVIEW QUESTION / Q008

How do git diff, git diff --staged and git diff HEAD differ?

THE SHORT ANSWER

Plain diff compares working files to the index; staged diff compares the index to HEAD; diff HEAD compares tracked working content to HEAD.

COMMAND EXAMPLE / SCENARIO

```
git diff -- src/Order.java
   git diff --staged -- src/Order.java
   git diff HEAD -- src/Order.java
```

✎ REMEMBER git diff -- src/Orderjava

WHAT REALLY HAPPENS

The third comparison can include both staged and unstaged effects, but ordinary diff does not display an untracked file as a normal tracked change. Use the comparison that answers your actual question: what remains unstaged, or what the next commit would contain.

FOLLOW-UP QUESTION + ANSWER

Which command reviews the proposed commit before committing?

`git diff --staged` shows the exact tracked changes currently proposed for the next commit.

⚠ **INTERVIEW TRAP:** Plain git diff can be empty even when staged changes exist.

Original question & example: uploaded Git guide, source page 8. Follow-up answer: added for this edition.

CHAPTER 02 / Working tree, index and staging Q 009 of 110

INTERVIEW QUESTION / Q009

How can you stage only part of a file?

THE SHORT ANSWER

Use interactive patch staging to select relevant hunks, then review the staged diff. Split hunks or edit the patch when the changes need finer separation.

COMMAND EXAMPLE / SCENARIO

```
git add -p -- src/OrderService.java
   git diff --staged
   git diff
```

✎ **REMEMBER** git add -p -- src/OrderService.java

WHAT REALLY HAPPENS

The index may now contain a different version from both HEAD and the working file. Test the intended staged snapshot or separate changes carefully so the commit is coherent. New files may need intent-to-add before patch selection.

FOLLOW-UP QUESTION + ANSWER

How do you keep a formatting change out of a bug-fix commit?

Stage only the bug-fix hunks with `git add -p`, review with `git diff --staged`, and commit formatting separately.

⚠ **INTERVIEW TRAP:** Staging one hunk does not guarantee the resulting commit compiles independently.

Original question & example: uploaded Git guide, source page 8. Follow-up answer: added for this edition.

CHAPTER 02 / Working tree, index and staging Q 010 of 110

INTERVIEW QUESTION / Q010

How do you unstage a file without deleting the edit?

THE SHORT ANSWER

Restore the index entry from HEAD while leaving the working tree alone. This changes what the next commit contains, not the edited file itself.

COMMAND EXAMPLE / SCENARIO

```
git restore --staged -- src/Order.java
   git diff --staged
   git diff -- src/Order.java
```

✎ **REMEMBER** git restore --staged -- src/Order.java

WHAT REALLY HAPPENS

This assumes the repository already has a HEAD commit. Older workflows use path-form reset for the same index operation. Verify the remaining working diff afterward.

FOLLOW-UP QUESTION + ANSWER

How does unstaging differ from restoring the working file?

Unstaging changes the index but keeps the edited file. `git restore -- FILE` replaces the unstaged tracked file content and can lose edits.

⚠ **INTERVIEW TRAP:** Adding --worktree as well changes the files and can discard edits.

Original question & example: uploaded Git guide, source page 8. Follow-up answer: added for this edition.

CHAPTER 03 / Setup, configuration and cloning Q 011 of 110

INTERVIEW QUESTION / Q011

How do git init and git clone differ?

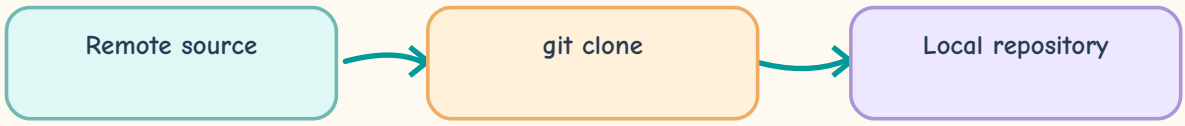
THE SHORT ANSWER

Init creates a repository in a chosen directory. Clone creates a repository from another one, normally configuring a remote and checking out an initial branch.

COMMAND EXAMPLE / SCENARIO

```
git init -b main interview-demo
   git clone REPOSITORY_URL order-service
   git -C order-service remote -v
```

DRAW IT TO REMEMBER



Identity settings and network authentication differ.

WHAT REALLY HAPPENS

An existing folder can be initialized, but its files are not committed automatically. A normal clone creates local and remote-tracking references according to the remote configuration. A bare clone is different because it has no ordinary checked-out working tree.

FOLLOW-UP QUESTION + ANSWER

Which remote is normally configured by clone?

Normally `origin`. Confirm with `git remote -v`; it is only a conventional name.

⚠ **INTERVIEW TRAP:** A folder containing source files is not versioned until those files are tracked and committed.

Original question & example: uploaded Git guide, source page 9. Follow-up answer: added for this edition.

CHAPTER 03 / Setup, configuration and cloning Q 012 of 110

INTERVIEW QUESTION / Q012

What is the difference between local, global and system Git configuration?

THE SHORT ANSWER

Local configuration applies to one repository, global to a user, and system to a machine installation. Higher-precedence sources can override earlier settings; inspect the origin when behavior is surprising.

COMMAND EXAMPLE / SCENARIO

```
git config --local user.name "Interview Learner"
   git config --local user.email "learner@example.test"
   git config --show-origin --get-all user.email
```

✎ REMEMBER git config --local user.name "Interview Learner"

WHAT REALLY HAPPENS

Command-line settings and environment variables can also affect behavior. Repository-local identity is useful when work and personal projects differ. Avoid assuming that the global value is the effective value in every repository.

FOLLOW-UP QUESTION + ANSWER

How would you find where pull.rebase was configured?

Use `git config --show-origin --show-scope --get-all pull.rebase` to see the effective values and their configuration sources.

⚠ **INTERVIEW TRAP:** Changing a global setting can alter unrelated repositories for that user.

Original question & example: uploaded Git guide, source page 9. Follow-up answer: added for this edition.

CHAPTER 03 / Setup, configuration and cloning Q 013 of 110

INTERVIEW QUESTION / Q013

Does user.name or user.email authenticate a push?

THE SHORT ANSWER

No. They normally populate commit identity metadata. Network authentication uses the transport and credential mechanism, such as an SSH key or an HTTPS credential helper.

COMMAND EXAMPLE / SCENARIO

```
git config user.email "developer@example.test"
   git remote get-url origin
   git config --get credential.helper
```

✎ REMEMBER git config user.email "developer@example.test"

WHAT REALLY HAPPENS

An author email can be configured locally without proving control of that address. Hosting platforms may associate verified emails or signatures with accounts, but authorization to update a repository is separate. Commit signing also differs from transport authentication.

FOLLOW-UP QUESTION + ANSWER

Why can committing work while pushing returns permission denied?

Local commits need no remote permission. Pushing requires network access and valid authentication plus permission to update the target reference.

⚠ **INTERVIEW TRAP:** A successful local commit does not show that you can push to the server.

Original question & example: uploaded Git guide, source page 10. Follow-up answer: added for this edition.

CHAPTER 03 / Setup, configuration and cloning Q 014 of 110

INTERVIEW QUESTION / Q014

Why should you explicitly select the initial branch name in examples?

THE SHORT ANSWER

The initial name can depend on Git configuration or hosting conventions. Explicit selection makes a training repository repeatable without changing a user's global defaults.

COMMAND EXAMPLE / SCENARIO

```
git init -b main interview-demo
   git config --get init.defaultBranch
   git branch --show-current
```

✎ **REMEMBER** git init -b main interview-demo

WHAT REALLY HAPPENS

Main and master are names, not different Git branch types. Renaming a default branch in a team also affects server settings, CI, documentation and clones. In this guide, main is a deliberate local-lab convention.

FOLLOW-UP QUESTION + ANSWER

What systems must be updated when a team renames its default branch?

Update the hosting default branch and branch protection, CI triggers, open PR targets, deployment scripts, docs and developer clones/upstreams.

⚠ **INTERVIEW TRAP:** Renaming locally does not automatically update the server's default branch setting.

Original question & example: uploaded Git guide, source page 10. Follow-up answer: added for this edition.

CHAPTER 03 / Setup, configuration and cloning Q 015 of 110

INTERVIEW QUESTION / Q015

What is a bare repository used for?

THE SHORT ANSWER

A bare repository stores Git data without an ordinary working tree. It is commonly used as a remote endpoint because branches can be updated without overwriting a checked-out application folder.

COMMAND EXAMPLE / SCENARIO

```
git init --bare demo-remote.git
   git clone demo-remote.git developer-a
   git -C developer-a remote -v
```

✎ REMEMBER git init --bare demo-remote.git

WHAT REALLY HAPPENS

The labs use local bare repositories to demonstrate sharing without contacting a hosting service. A bare repository is not automatically secured, backed up or configured with review policy; those concerns belong to its deployment.

FOLLOW-UP QUESTION + ANSWER

Why is pushing to a checked-out non-bare branch restricted by default?

The push could change the branch's files without updating the remote working tree/index. Bare repositories avoid that mismatch.

⚠ **INTERVIEW TRAP:** A bare remote does not provide a pull-request interface by itself.

Original question & example: uploaded Git guide, source page 10. Follow-up answer: added for this edition.

CHAPTER 04 / Commits, inspection and useful history queries Q 016 of 110

INTERVIEW QUESTION / Q016

What makes a good commit?

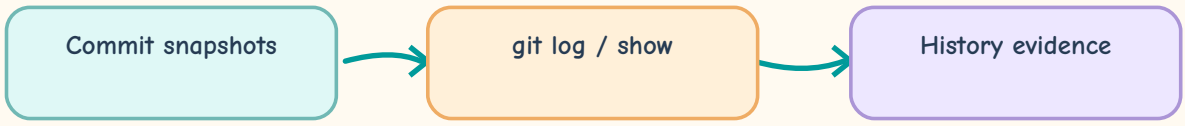
THE SHORT ANSWER

A good commit captures one understandable purpose, includes necessary tests or configuration and has a message explaining the behavior change. Keep unrelated formatting and fixes separate when practical.

COMMAND EXAMPLE / SCENARIO

```
git diff --staged
   git commit -m "Reject negative checkout quantity"
   git show --stat HEAD
```

DRAW IT TO REMEMBER



Parents define history; display order is a view.

WHAT REALLY HAPPENS

The commit should be reviewable and useful for later debugging or revert. A small commit is not necessarily coherent if it relies on an uncommitted dependency. Explain why the change is needed in the message body when the diff alone is insufficient.

FOLLOW-UP QUESTION + ANSWER

Can this commit be reverted without removing an unrelated feature?

Usually yes if the commit contains only one coherent change. `git revert OID` then test; unrelated later commits remain in history.

⚠ **INTERVIEW TRAP:** git commit -a includes tracked changes, but does not automatically add untracked files.

Original question & example: uploaded Git guide, source page 11. Follow-up answer: added for this edition.

CHAPTER 04 / Commits, inspection and useful history queries Q 017 of 110

INTERVIEW QUESTION / Q017

How do you inspect one commit and its changed files?

THE SHORT ANSWER

Use show to inspect the commit patch and metadata, or request summary forms when only paths and sizes matter.

COMMAND EXAMPLE / SCENARIO

```
git show HEAD
   git show --name-status HEAD
   git show --stat HEAD
   git show HEAD:src/Order.java
```

✎ REMEMBER git show HEAD

WHAT REALLY HAPPENS

The last command reads a path from the selected commit without switching branches. Merge commits have multiple-parent comparison semantics, so choose the desired parent comparison when investigating a merge.

FOLLOW-UP QUESTION + ANSWER

How can you inspect yesterday's version without changing your files?

Use `git show COMMIT:path/to/File.java` or `git show HEAD@{yesterday}:path` if that reflog date exists. These commands do not change your files.

⚠ **INTERVIEW TRAP:** A file path must exist in the selected snapshot, not just in today's working tree.

Original question & example: uploaded Git guide, source page 11. Follow-up answer: added for this edition.

CHAPTER 04 / Commits, inspection and useful history queries Q 018 of 110

INTERVIEW QUESTION / Q018

How do you view branches and merges in history?

THE SHORT ANSWER

Use a graph-oriented log with reference decoration and a bounded scope. Read parent relationships rather than treating screen order as a universal chronological sequence.

COMMAND EXAMPLE / SCENARIO

```
git log --graph --oneline --decorate --all
   git log --first-parent --oneline main
```

✎ REMEMBER git log --graph --oneline --decorate --all

WHAT REALLY HAPPENS

The first view shows reachable history across selected references. First-parent follows the main line through merges and can help understand integration history. Author timestamps and topology can disagree, so a log is not a simple numbered timeline.

FOLLOW-UP QUESTION + ANSWER

Why might a side-branch commit appear below a newer main commit?

Git graphs follow parent relationships, not just timestamps; branch topology and display ordering can place commits in surprising rows.

⚠ **INTERVIEW TRAP:** A commit missing from a limited log view may still exist elsewhere in the repository.

Original question & example: uploaded Git guide, source page 12. Follow-up answer: added for this edition.

CHAPTER 04 / Commits, inspection and useful history queries Q 019 of 110

INTERVIEW QUESTION / Q019

What do A..B and A...B mean in git log?

THE SHORT ANSWER

Two-dot selects commits reachable from B but not A. Three-dot selects commits reachable from either side but not both: the symmetric difference.

COMMAND EXAMPLE / SCENARIO

```
git log main..feature
   git log --left-right main...feature
   git rev-list --left-right --count main...feature
```

✎ REMEMBER git log main..feature

WHAT REALLY HAPPENS

These are set operations over commit ancestry. The notation has different comparison meaning in git diff: three-dot diff compares the merge base to the second endpoint. Name the command when explaining the range.

FOLLOW-UP QUESTION + ANSWER

Which range shows feature commits not reachable from main?

`git log main..feature` lists commits reachable from `feature` but not from `main`.

⚠ **INTERVIEW TRAP:** Log three-dot and diff three-dot are not interchangeable definitions.

Original question & example: uploaded Git guide, source page 12. Follow-up answer: added for this edition.

CHAPTER 04 / Commits, inspection and useful history queries Q 020 of 110

INTERVIEW QUESTION / Q020

How do you compare a feature's changes with main?

THE SHORT ANSWER

Use a merge-base comparison to inspect what the feature changed since its shared ancestor, and an endpoint comparison when you want final snapshot differences.

COMMAND EXAMPLE / SCENARIO

```
git diff main...feature
   git diff main feature
   git merge-base main feature
```

✎ REMEMBER git diff main…feature

WHAT REALLY HAPPENS

The first comparison is useful for many review workflows because it excludes independent main changes from the starting snapshot. The second answers how the two tips differ now. Neither view alone proves that the feature integrates cleanly with current main.

FOLLOW-UP QUESTION + ANSWER

What changes if main advanced after the feature branch was created?

The two branch tips may now differ in more ways, and merging may conflict. `git diff main…feature` still starts from their merge base.

⚠ **INTERVIEW TRAP:** A clean-looking feature diff can still conflict during integration.

Original question & example: uploaded Git guide, source page 12. Follow-up answer: added for this edition.

CHAPTER 05 / Branches, switching and detached HEAD Q 021 of 110

INTERVIEW QUESTION / Q021

How do you create and switch to a branch?

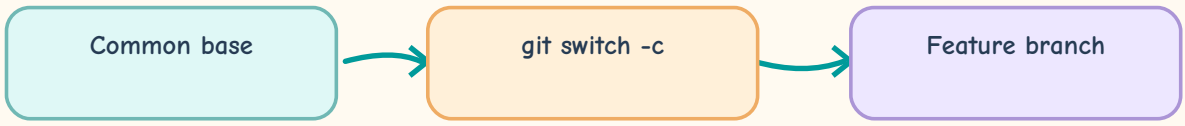
THE SHORT ANSWER

Use switch -c to create a branch at the current commit and check it out. Use switch with an existing name to change the checked-out branch.

COMMAND EXAMPLE / SCENARIO

```
git switch -c feature/order-validation
   git branch --show-current
   git switch main
```

DRAW IT TO REMEMBER



A branch is a movable reference, not a copied folder.

WHAT REALLY HAPPENS

Creating the branch does not create a new commit. Both branch names can initially point to the same commit. Subsequent commits on the feature move only that branch. Switching may be refused if local edits would be overwritten.

FOLLOW-UP QUESTION + ANSWER

What happens to main when you commit on the new feature?

Nothing moves on `main`. Your newly created commit advances only the currently checked-out feature branch.

⚠ **INTERVIEW TRAP:** A successful switch can carry compatible uncommitted edits into the new branch.

Original question & example: uploaded Git guide, source page 13. Follow-up answer: added for this edition.

CHAPTER 05 / Branches, switching and detached HEAD Q 022 of 110

INTERVIEW QUESTION / Q022

How do git switch and git checkout differ?

THE SHORT ANSWER

Switch focuses on selecting branches or detached commits. Checkout is an older multipurpose command that can also replace paths. Restore separates file-restoration operations from branch switching.

COMMAND EXAMPLE / SCENARIO

```
git switch main
   git switch --detach v1.0.0
   git restore --source=HEAD -- src/Order.java
```

✎ REMEMBER git switch main

WHAT REALLY HAPPENS

Explicit commands reduce ambiguity between a branch name and a path. Switch and restore are available in modern Git; this guide's labs run on Git 2.51.1. Older installations may require the corresponding checkout forms.

FOLLOW-UP QUESTION + ANSWER

Why is git checkout ambiguous when a path and branch share a name?

Git cannot always tell if that word names a file or a branch. Use `git switch branch` or `git restore -- file` to make intent clear.

⚠ INTERVIEW TRAP: Checkout of a path changes file content; it is not a branch switch.

Original question & example: uploaded Git guide, source page 14. Follow-up answer: added for this edition.

CHAPTER 05 / Branches, switching and detached HEAD Q 023 of 110

INTERVIEW QUESTION / Q023

What is detached HEAD and is it always a problem?

THE SHORT ANSWER

Detached HEAD points directly to a commit instead of a branch. It is useful for inspecting or testing an old version, but new work needs a reference if you want to keep it reachable.

COMMAND EXAMPLE / SCENARIO

```
git switch --detach v1.0.0
   # after creating an experimental commit:
   git switch -c experiment/fix
```

✎ REMEMBER git switch --detach v1.0.0

WHAT REALLY HAPPENS

Creating a branch at the detached tip preserves the new commits. If you already switched away, inspect the reflog and create a branch at the intended commit. CI often checks out a detached commit intentionally for reproducible builds.

FOLLOW-UP QUESTION + ANSWER

How do you keep an experiment before returning to main?

Before leaving detached HEAD, run `git switch -c experiment/name` at the experiment tip. That branch keeps the commit reachable.

⚠ **INTERVIEW TRAP:** Detached commits are not immediately deleted, but unreferenced history can eventually be collected.

Original question & example: uploaded Git guide, source page 14. Follow-up answer: added for this edition.

CHAPTER 05 / Branches, switching and detached HEAD Q 024 of 110

INTERVIEW QUESTION / Q024

How do you rename or delete a branch?

THE SHORT ANSWER

Rename updates the local branch name. Delete removes a local branch reference, with a checked deletion policy or a forced form. Remote references require separate operations.

COMMAND EXAMPLE / SCENARIO

```
git branch -m feature/old-name feature/new-name
   git branch -d feature/done
   git push origin --delete feature/done
```

✎ REMEMBER git branch -m feature/old-name feature/new-name

WHAT REALLY HAPPENS

The remote delete line changes the server and needs the team's authorization. Local -d checks merge reachability using its documented rules, not whether a squash-merged patch looks equivalent. Renaming may require updating upstream and hosting settings.

FOLLOW-UP QUESTION + ANSWER

Why might -d reject a feature that was squash-merged?

Squash creates a different commit on main, without preserving original feature commits as ancestors. `-d` checks reachability, not patch similarity.

⚠ INTERVIEW TRAP: Branch deletion removes a name, not necessarily all reachable commit objects.

Original question & example: uploaded Git guide, source page 15. Follow-up answer: added for this edition.

CHAPTER 05 / Branches, switching and detached HEAD Q 025 of 110

INTERVIEW QUESTION / Q025

Why can uncommitted changes follow you when switching branches?

THE SHORT ANSWER

Edits live in the working tree and index, not in an immutable branch snapshot. Git may carry them when the switch can preserve them without overwriting incompatible content.

COMMAND EXAMPLE / SCENARIO

```
git status --short
   git switch feature/another-task
   git status --short
```

✎ REMEMBER git status --short

WHAT REALLY HAPPENS

Do not rely on accidental carry-over when changing tasks. Commit coherent work, stash it with the needed file categories, or use another worktree. If a switch is refused, inspect rather than forcing away useful edits.

FOLLOW-UP QUESTION + ANSWER

How would you move an unfinished edit to a separate task safely?

Commit it on a dedicated WIP branch, or `git stash push -u` then create/switch to the other branch and apply the stash deliberately.

⚠ **INTERVIEW TRAP:** Forcing a checkout or switch can discard local changes.

Original question & example: uploaded Git guide, source page 15. Follow-up answer: added for this edition.

CHAPTER 06 / Merging and resolving conflicts Q 026 of 110

INTERVIEW QUESTION / Q026

What is a fast-forward merge?

THE SHORT ANSWER

When the current tip is an ancestor of the target tip, integration can move the current branch reference forward without creating a merge commit.

COMMAND EXAMPLE / SCENARIO

```
git switch main
   git merge --ff-only feature/orders
   git log --oneline --decorate -3
```

DRAW IT TO REMEMBER



Conflicts need a correct final result and tests.

WHAT REALLY HAPPENS

The existing feature commits become reachable from main. No new two-parent commit is required. The --ff-only policy refuses integration if histories have diverged, making the allowed outcome explicit.

FOLLOW-UP QUESTION + ANSWER

What happens if main already contains an independent commit?

`--ff-only` fails because neither tip is the ancestor of the other. Use an agreed merge or rebase after inspecting the diverged history.

⚠ **INTERVIEW TRAP:** Fast-forward does not mean changes were tested or reviewed.

Original question & example: uploaded Git guide, source page 16. Follow-up answer: added for this edition.

CHAPTER 06 / Merging and resolving conflicts Q 027 of 110

INTERVIEW QUESTION / Q027

What is a three-way merge and a merge commit?

THE SHORT ANSWER

A three-way merge uses a common ancestor and both tips to combine changes. When divergent histories are integrated normally, a merge commit records both parents.

COMMAND EXAMPLE / SCENARIO

```
git switch main
   git merge feature/orders
   git show --no-patch --format=%P HEAD
```

✎ REMEMBER git switch main

WHAT REALLY HAPPENS

Two parents preserve ancestry from both lines of work. The first parent is normally the branch that was checked out when merging. A merge can be conflict-free when both sides change different files, yet still require integration testing.

FOLLOW-UP QUESTION + ANSWER

Why does preserving both parents matter for future merges?

Git remembers both lines as ancestors, so future merges can identify common commits and avoid re-merging already integrated history.

⚠ **INTERVIEW TRAP:** A merge commit can include a hand-written resolution beyond either parent.

Original question & example: uploaded Git guide, source page 17. Follow-up answer: added for this edition.

CHAPTER 06 / Merging and resolving conflicts Q 028 of 110

INTERVIEW QUESTION / Q028

How do you resolve a merge conflict correctly?

THE SHORT ANSWER

Inspect the conflicting intent, edit the final desired content, stage the resolution, verify it and continue the merge. Do not choose a side solely to remove markers.

COMMAND EXAMPLE / SCENARIO

```
git status
   git diff --cc
   # edit file and remove conflict markers after deciding the result
   git add -- src/Order.java
   git merge --continue
```

✎ REMEMBER git status

WHAT REALLY HAPPENS

Staging records the chosen resolved version. Run relevant tests because a syntactically valid combination may violate behavior. Deleted/modified or renamed paths need a decision about the path as well as the content.

FOLLOW-UP QUESTION + ANSWER

Which test demonstrates that both branches' intended behavior survives?

Run unit and integration tests for the feature introduced by each branch, plus one test of the combined workflow after conflict resolution.

⚠ **INTERVIEW TRAP:** Conflict markers left in Java files can become compile errors or accidental text.

Original question & example: uploaded Git guide, source page 17. Follow-up answer: added for this edition.

CHAPTER 06 / Merging and resolving conflicts Q 029 of 110

INTERVIEW QUESTION / Q029

How do --no-ff, --ff-only and --squash differ?

THE SHORT ANSWER

No-ff creates a merge commit even when a fast-forward is possible. Ff-only allows only a fast-forward. Squash prepares combined changes for a later ordinary commit without recording merge ancestry.

COMMAND EXAMPLE / SCENARIO

```
git merge --no-ff feature
   git merge --ff-only feature
   git merge --squash feature
   git commit -m "Add order validation"  # after squash
```

✎ REMEMBER git merge --no-ff feature

WHAT REALLY HAPPENS

These are alternatives, not commands to run consecutively for the same integration. Squash is useful for one reviewable result but changes how later Git operations recognize ancestry. Choose the repository's integration policy rather than using a flag mechanically.

FOLLOW-UP QUESTION + ANSWER

Why can later merging the same long-lived branch become surprising after squash?

The squash patch is present, but the old feature commits are not ancestors. A later merge may replay changes or produce unexpected conflicts.

⚠ **INTERVIEW TRAP:** Squash merge does not mark all original feature commits as ancestors of main.

Original question & example: uploaded Git guide, source page 18. Follow-up answer: added for this edition.

CHAPTER 06 / Merging and resolving conflicts Q 030 of 110

INTERVIEW QUESTION / Q030

How do you abort a merge, and what are the limits?

THE SHORT ANSWER

Merge --abort attempts to restore the pre-merge state. Start with a clean working tree or preserve existing edits because reconstructing them after a complex interrupted merge is not always guaranteed.

COMMAND EXAMPLE / SCENARIO

```
git status
   git merge --abort
   git status
```

✎ REMEMBER git status

WHAT REALLY HAPPENS

Abort is for an in-progress merge, not for removing an already committed merge. After a committed shared merge, a revert may be appropriate. Verify that your original edits are present instead of assuming every cancellation has identical behavior.

FOLLOW-UP QUESTION + ANSWER

What should you do if the merge already became a public commit?

Do not reset the shared branch to erase history. Consider `git revert -m` for a public merge, confirm the right mainline parent, and review/test the result.

⚠ **INTERVIEW TRAP:** Mixing unrelated local edits with conflict resolution makes recovery harder.

Original question & example: uploaded Git guide, source page 18. Follow-up answer: added for this edition.

CHAPTER 07 / Rebase and rewriting a private branch Q 031 of 110

INTERVIEW QUESTION / Q031

What does git rebase do?

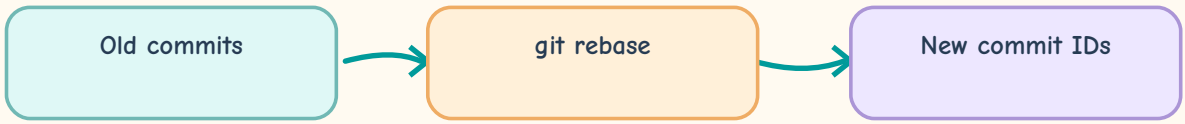
THE SHORT ANSWER

Rebase takes selected commits from one line of development and replays their changes onto another base, normally producing new commit identities and moving the rebased branch tip.

COMMAND EXAMPLE / SCENARIO

```
git switch feature/orders
   git branch backup/feature-before-rebase
   git rebase main
```

DRAW IT TO REMEMBER



Rewrite only with clear collaboration rules.

WHAT REALLY HAPPENS

Use the backup reference to preserve the original tip while learning or doing a risky rewrite. Main is not moved by rebasing the feature onto it. Replay can omit commits already represented upstream according to Git's rules.

FOLLOW-UP QUESTION + ANSWER

Which branch moves when you rebase feature onto main?

The checked-out `feature` branch tip moves to new replayed commits. The `main` branch tip does not move.

⚠ **INTERVIEW TRAP:** Rebase is not simply moving unchanged commit objects to a different parent.

Original question & example: uploaded Git guide, source page 19. Follow-up answer: added for this edition.

CHAPTER 07 / Rebase and rewriting a private branch Q 032 of 110

INTERVIEW QUESTION / Q032

When should you merge and when should you rebase?

THE SHORT ANSWER

Merge preserves the existing ancestry and integrates it. Rebase can simplify a private branch before integration, but rewriting shared commits requires coordination with people depending on them.

COMMAND EXAMPLE / SCENARIO

```
Shared integration branch: preserve accepted history.
   Private feature: fetch, then rebase onto selected base if policy permits.
   Release hotfix: choose explicit backport or merge strategy.
```

✎ **REMEMBER** Shared integration branch: preserve accepted history.

WHAT REALLY HAPPENS

There is no universally superior history shape. Merge records integration events; a linear history can make some navigation easier. Focus on audit, review and collaboration requirements rather than visual neatness alone.

FOLLOW-UP QUESTION + ANSWER

What team agreement would make a shared rebase acceptable?

Agree who may rewrite, when teammates must stop working, how backups/notifications work, and how to synchronize; protect the remote update with an explicit lease.

⚠ **INTERVIEW TRAP:** Rebasing a published branch can leave coworkers based on commits you replaced.

Original question & example: uploaded Git guide, source page 20. Follow-up answer: added for this edition.

CHAPTER 07 / Rebase and rewriting a private branch Q 033 of 110

INTERVIEW QUESTION / Q033

How do you resolve a rebase conflict?

THE SHORT ANSWER

Resolve the current replayed change, stage the desired files and continue. Abort returns to the original branch state; skip drops the current replayed commit's contribution and needs careful review.

COMMAND EXAMPLE / SCENARIO

```
git status
   # edit conflicting file
   git add -- src/Order.java
   git rebase --continue
   # alternative cancellation: git rebase --abort
```

✎ REMEMBER git status

WHAT REALLY HAPPENS

Several commits can conflict in sequence, so resolving the first does not necessarily finish the operation. Rebase --skip is not a generic fix for a difficult conflict. Check the final diff and tests against the intended feature.

FOLLOW-UP QUESTION + ANSWER

How do you determine whether a skipped change is actually redundant?

Compare the original patch with upstream using `git show`/`git range-diff`; skip only after confirming the intended behavior already exists, then run tests.

⚠ **INTERVIEW TRAP:** Skipping a commit can silently remove behavior if its change was not already present.

Original question & example: uploaded Git guide, source page 20. Follow-up answer: added for this edition.

CHAPTER 07 / Rebase and rewriting a private branch Q 034 of 110

INTERVIEW QUESTION / Q034

Why can ours and theirs feel reversed during rebase?

THE SHORT ANSWER

During a rebase merge-backend conflict, ours is the already-rebased side including the new base, while theirs is the commit being replayed from your old branch.

COMMAND EXAMPLE / SCENARIO

```
git ls-files -u
   git show :2:src/Order.java  # stage 2: ours
   git show :3:src/Order.java  # stage 3: theirs
```

✎ REMEMBER git ls-files -u

WHAT REALLY HAPPENS

These labels describe the current operation, not permanently your work versus a colleague's. In a normal merge they refer to the checked-out side and the other side. Inspect the content and intent before selecting a side.

FOLLOW-UP QUESTION + ANSWER

What does stage 1 represent in a conflicted index?

Stage 1 stores the common base version of a conflicted path; stages 2 and 3 store the two competing versions.

⚠ **INTERVIEW TRAP:** Blindly using --ours to keep your feature during rebase can select the opposite content.

Original question & example: uploaded Git guide, source page 21. Follow-up answer: added for this edition.

CHAPTER 07 / Rebase and rewriting a private branch Q 035 of 110

INTERVIEW QUESTION / Q035

How do you review a branch after rebase?

THE SHORT ANSWER

Compare the rewritten patch series with the preserved old series, inspect the final base-relative diff and run relevant tests. New IDs alone are expected and do not indicate changed behavior.

COMMAND EXAMPLE / SCENARIO

```
git range-diff main..backup/feature-before-rebase main..feature/orders
   git diff main...feature/orders
   git log --graph --oneline --decorate -10
```

✎ REMEMBER git range-diff main..backup/feature-before-rebase main..feature/orders

WHAT REALLY HAPPENS

Choose the old and new base ranges accurately; the first line is a convenient sketch when main is a suitable boundary for both. Range-diff highlights patch-series changes but is not a correctness proof. Check conflict resolutions particularly closely.

FOLLOW-UP QUESTION + ANSWER

How would you distinguish expected ID changes from unexpected patch changes?

Use `git range-diff` between original and rebased series. New hashes are expected; unexpected added, removed or changed patches need review.

⚠ INTERVIEW TRAP: A conflict resolution can unintentionally delete part of the old feature.

Original question & example: uploaded Git guide, source page 21. Follow-up answer: added for this edition.

CHAPTER 08 / Cherry-pick, amend and interactive cleanup Q 036 of 110

INTERVIEW QUESTION / Q036

What does cherry-pick do?

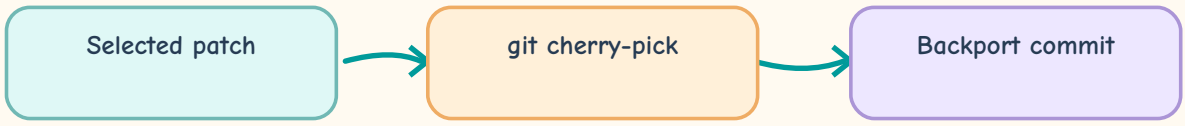
THE SHORT ANSWER

> Cherry-pick applies the change introduced by a selected commit to the current branch, usually creating a new commit there. It does not merge the entire source branch's ancestry.

COMMAND EXAMPLE / SCENARIO

```
git switch release/1.4
   git cherry-pick -x FIX_COMMIT
   git show --stat HEAD
```

DRAW IT TO REMEMBER



Transferred changes need not share ancestry.

WHAT REALLY HAPPENS

The -x trailer records the source commit for a backport. The fix may depend on earlier source-branch changes, so inspect prerequisites and test the release version. Identical patches can have different IDs because parent and metadata differ.

FOLLOW-UP QUESTION + ANSWER

How would you backport a fix that depends on a renamed method?

Backport any necessary rename first, or adapt the patch to the old method name on the release branch; compile and test against that release API.

⚠ **INTERVIEW TRAP:** Selecting one commit does not automatically select its semantic dependencies.

Original question & example: uploaded Git guide, source page 22. Follow-up answer: added for this edition.

CHAPTER 08 / Cherry-pick, amend and interactive cleanup Q 037 of 110

INTERVIEW QUESTION / Q037

How do you handle a cherry-pick conflict?

THE SHORT ANSWER

Resolve and stage the current change, then continue the cherry-pick. Abort abandons the in-progress sequence; skip deliberately omits the current selected commit.

COMMAND EXAMPLE / SCENARIO

```
git status
   git add -- src/Order.java
   git cherry-pick --continue
   # to abandon: git cherry-pick --abort
```

✎ REMEMBER git status

WHAT REALLY HAPPENS

Read the source patch and destination behavior rather than copying the source file wholesale. When picking multiple commits, several may need resolution. A conflict-free backport can still compile incorrectly against older dependencies.

FOLLOW-UP QUESTION + ANSWER

Which release-specific tests should you run after the backport?

Run the release's own build/tests, regression tests for the defect, and compatibility tests for its Java version, dependencies and schema.

⚠ **INTERVIEW TRAP:** Choosing the whole source version may bring unrelated newer behavior into the release.

Original question & example: uploaded Git guide, source page 23. Follow-up answer: added for this edition.

CHAPTER 08 / Cherry-pick, amend and interactive cleanup Q 038 of 110

INTERVIEW QUESTION / Q038

What does git commit --amend change?

THE SHORT ANSWER

Amend creates a replacement for the current tip using the selected staged content and message options. The old commit is not edited in place, and its ID usually changes.

COMMAND EXAMPLE / SCENARIO

```
git add -- src/OrderTest.java
   git commit --amend --no-edit
   git show --stat HEAD
```

✎ REMEMBER git add -- src/OrderTest.java

WHAT REALLY HAPPENS

No-edit keeps the message; it does not mean keeping the original content. For a message-only amendment, ensure no unintended staged changes are present. The default author handling differs from the updated committer metadata.

FOLLOW-UP QUESTION + ANSWER

How would you change only the message without adding staged edits?

First check `git diff --staged` and unstage anything unintended. Then `git commit --amend -m "New message"` on a private commit.

⚠ INTERVIEW TRAP: Amending a shared tip is a history rewrite, not an ordinary new follow-up commit.

Original question & example: uploaded Git guide, source page 23. Follow-up answer: added for this edition.

CHAPTER 08 / Cherry-pick, amend and interactive cleanup Q 039 of 110

INTERVIEW QUESTION / Q039

What can interactive rebase do?

THE SHORT ANSWER

It lets you reorder, reword, combine, edit or drop selected commits while replaying the series. The selected range must match what you intend to rewrite.

COMMAND EXAMPLE / SCENARIO

```
git branch backup/cleanup
   git rebase -i HEAD~3
   # todo actions: pick, reword, edit, squash, fixup, drop
```

✎ REMEMBER git branch backup/cleanup

WHAT REALLY HAPPENS

Squash combines changes and lets you compose a message; fixup normally discards the fixup message. Reordering can break dependencies and dropping removes a change from the result. Keep a reference to the original and inspect the final series.

FOLLOW-UP QUESTION + ANSWER

How would you split one large commit into two coherent changes?

Run interactive rebase with `edit` on that commit, `git reset HEAD^`, stage the first logical part and commit, then stage and commit the remainder; continue rebase.

⚠ **INTERVIEW TRAP:** Interactive cleanup can produce intermediate commits that do not compile.

Original question & example: uploaded Git guide, source page 23. Follow-up answer: added for this edition.

CHAPTER 08 / Cherry-pick, amend and interactive cleanup Q 040 of 110

INTERVIEW QUESTION / Q040

How do fixup commits and autosquash help a review workflow?

THE SHORT ANSWER

A fixup commit identifies the earlier commit it should correct, and autosquash arranges it in an interactive rebase. This preserves review activity until the intentional cleanup step.

COMMAND EXAMPLE / SCENARIO

```
git add -- src/OrderTest.java
   git commit --fixup=TARGET_COMMIT
   git rebase -i --autosquash BASE_COMMIT
```

✎ REMEMBER git add -- src/OrderTest.java

WHAT REALLY HAPPENS

Choose a base before the target commit so both target and fixup appear in the todo list. This still rewrites commits and must follow the team's policy. Verify that the fixup attached to the intended change, especially when messages are similar.

FOLLOW-UP QUESTION + ANSWER

What happens if the target commit is outside the selected rebase range?

Autosquash cannot match a target absent from the todo list. Select an earlier base that includes the target and its fixup, then review the plan.

⚠ **INTERVIEW TRAP:** Autosquash is not automatically safe for a branch others are already using.

Original question & example: uploaded Git guide, source page 24. Follow-up answer: added for this edition.

CHAPTER 09 / Restore, reset and revert without confusion Q 041 of 110

INTERVIEW QUESTION / Q041

How do restore, reset and revert differ?

THE SHORT ANSWER

Restore replaces selected file/index content. Commit-form reset moves the current branch/HEAD and optionally changes index/files. Revert creates a new inverse commit without removing the original history.

COMMAND EXAMPLE / SCENARIO

```
Unstage: git restore --staged -- Order.java
   Discard unstaged tracked edit: git restore -- Order.java
   Undo a shared change: git revert BAD_COMMIT
```

DRAW IT TO REMEMBER



Safe undo keeps shared ancestry visible.

WHAT REALLY HAPPENS

Restore of the working file can discard edits, so inspect them first. Revert is normally the suitable starting point for a published change because existing ancestry remains. Reset is often useful for private cleanup but has multiple modes.

FOLLOW-UP QUESTION + ANSWER

Must the original commit remain visible to collaborators?

If collaborators already rely on it, usually yes: use `git revert` to undo while keeping shared history, rather than rewriting it.

⚠ **INTERVIEW TRAP:** The word undo does not identify which of these outcomes you need.

Original question & example: uploaded Git guide, source page 25. Follow-up answer: added for this edition.

CHAPTER 09 / Restore, reset and revert without confusion Q 042 of 110

INTERVIEW QUESTION / Q042

How do soft, mixed and hard reset affect state?

THE SHORT ANSWER

All three commit-form modes move the current branch/HEAD to the target. Soft leaves index/files; mixed resets the index but keeps files; hard resets both index and tracked files.

COMMAND EXAMPLE / SCENARIO

```
# Disposable repository only; inspect and preserve useful work first.
   git reset --soft HEAD~1
   git reset --mixed HEAD~1
   git reset --hard HEAD~1
```

✎ REMEMBER # Disposable repository only; inspect and preserve useful work first.

WHAT REALLY HAPPENS

These are alternative demonstrations, not a sequence for one task. Hard reset discards tracked local changes and can remove untracked items that obstruct writing tracked paths. It is not a general removal of every untracked file.

FOLLOW-UP QUESTION + ANSWER

Which mode keeps undone commit content staged?

`git reset --soft TARGET` moves the tip while leaving the old commit's changes in the index, ready to recommit.

⚠ **INTERVIEW TRAP:** Reflog may recover an old commit, but it does not restore arbitrary edits never recorded by Git.

Original question & example: uploaded Git guide, source page 26. Follow-up answer: added for this edition.

CHAPTER 09 / Restore, reset and revert without confusion Q 043 of 110

INTERVIEW QUESTION / Q043

How does path-form reset differ from commit-form reset?

THE SHORT ANSWER

A reset with paths changes selected index entries without moving the branch tip. A reset to a commit without paths can move the branch and apply a mode.

COMMAND EXAMPLE / SCENARIO

```
git reset HEAD -- src/Order.java
   git rev-parse HEAD
   git diff -- src/Order.java
```

✎ REMEMBER git reset HEAD -- src/Order.java

WHAT REALLY HAPPENS

The path form is an older way to unstage selected files. It leaves working content intact. A -- separator clearly distinguishes paths from revision arguments. Avoid applying the branch-moving mental model to every reset invocation.

FOLLOW-UP QUESTION + ANSWER

Why is HEAD unchanged after unstaging one path?

Path-form reset updates only chosen index entries. It does not move the current branch or HEAD commit.

⚠ **INTERVIEW TRAP:** You cannot use path-form reset to move only part of a branch history.

Original question & example: uploaded Git guide, source page 26. Follow-up answer: added for this edition.

CHAPTER 09 / Restore, reset and revert without confusion Q 044 of 110

INTERVIEW QUESTION / Q044

How do you revert a bad commit on a shared branch?

THE SHORT ANSWER

Inspect the commit and later dependencies, create an inverse commit, resolve any conflicts, test and publish through the team's review policy.

COMMAND EXAMPLE / SCENARIO

```
git show BAD_COMMIT
   git revert BAD_COMMIT
   git show --stat HEAD
```

✎ **REMEMBER** git show BAD_COMMIT

WHAT REALLY HAPPENS

Later code may depend on the removed behavior, so a mechanical inverse can still break the application. Reverting records a new history event and does not erase the original. Deploying an old artifact and reverting source are related but separate release actions.

FOLLOW-UP QUESTION + ANSWER

How would you revert one bug fix while preserving later unrelated changes?

Use `git revert BAD_COMMIT`, resolve any conflicts, and test. Revert adds an inverse commit instead of deleting later commits.

⚠ **INTERVIEW TRAP:** A successful revert command does not establish that the resulting application is correct.

Original question & example: uploaded Git guide, source page 27. Follow-up answer: added for this edition.

CHAPTER 09 / Restore, reset and revert without confusion Q 045 of 110

INTERVIEW QUESTION / Q045

Why does reverting a merge need a mainline parent?

THE SHORT ANSWER

A merge has multiple parents, so Git needs to know the parent relative to which the merge's change should be reversed. Inspect parent order before selecting it.

COMMAND EXAMPLE / SCENARIO

```
git show --no-patch --format=%P MERGE_COMMIT
   git revert -m 1 MERGE_COMMIT
```

✎ **REMEMBER** git show --no-patch --format=%P MERGE_COMMIT

WHAT REALLY HAPPENS

Parent 1 is commonly the previous checked-out branch tip, but verify it. The original merge remains an ancestor, so merging the same old branch again will not simply reapply all reverted changes. A later reintroduction needs an intentional strategy.

FOLLOW-UP QUESTION + ANSWER

Why might reverting the revert be needed before reintroducing the old feature?

Git still sees the old merge as integrated. Reverting the revert can restore its changes so future integration is deliberate; test for intervening changes.

⚠ **INTERVIEW TRAP:** Choosing -m 1 without checking the graph can undo the wrong side.

Original question & example: uploaded Git guide, source page 27. Follow-up answer: added for this edition.

CHAPTER 10 / Reflog, recovery and clean-up Q 046 of 110

INTERVIEW QUESTION / Q046

What is the reflog and how is it different from the log?

THE SHORT ANSWER

The log traverses commit ancestry. A reflog records local reference updates, including previous checkout or branch positions that may no longer be reachable from current branches.

COMMAND EXAMPLE / SCENARIO

```
git log --oneline -5
   git reflog show HEAD -10
   git reflog show main -10
```

DRAW IT TO REMEMBER



Preserve the old object before changing more refs.

WHAT REALLY HAPPENS

Reflogs are local and have expiration policies. They are not normally pushed to another clone. A reflog entry can help find the old tip after reset, rebase or a detached experiment, but it is not a permanent backup.

FOLLOW-UP QUESTION + ANSWER

Which view helps find a tip removed by a local reset?

`git reflog show HEAD` or the branch's reflog shows previous local tip positions even after a reset.

⚠ **INTERVIEW TRAP:** Your colleague's reflog is not automatically available in your clone.

Original question & example: uploaded Git guide, source page 28. Follow-up answer: added for this edition.

CHAPTER 10 / Reflog, recovery and clean-up Q 047 of 110

INTERVIEW QUESTION / Q047

How do you recover a commit after an accidental reset?

THE SHORT ANSWER

Find and verify the intended old commit, then create a rescue reference before doing further history changes. Recovering the branch can be a later deliberate step.

COMMAND EXAMPLE / SCENARIO

```
git reflog
   git show RECOVERED_OID
   git branch rescue/lost-work RECOVERED_OID
```

✎ REMEMBER git reflog

WHAT REALLY HAPPENS

A rescue branch makes the object reachable and lets you compare it safely with current work. The correct reflog selector depends on the operations performed; do not assume HEAD@{1} is always the lost commit. Preserve the current tip too if it contains useful changes.

FOLLOW-UP QUESTION + ANSWER

How do you confirm the rescued commit contains the expected files?

Inspect `git show --stat RECOVERED_OID`, `git ls-tree -r --name-only RECOVERED_OID` and the file's contents before moving branches.

⚠ **INTERVIEW TRAP:** Another reset before inspection can make a confused recovery even harder to reason about.

Original question & example: uploaded Git guide, source page 28. Follow-up answer: added for this edition.

CHAPTER 10 / Reflog, recovery and clean-up Q 048 of 110

INTERVIEW QUESTION / Q048

Can Git recover an uncommitted deleted edit?

THE SHORT ANSWER

> Only if that content was recorded somewhere Git can still find, such as a commit, stash or staged blob. Never-added working edits are generally outside Git's recovery history.

COMMAND EXAMPLE / SCENARIO

```
Check: commits, stash list, reflog, editor local history.
   If staged content became unreachable, object inspection may help.
   Use backups for content Git never stored.
```

✎ **REMEMBER** Check: commits, stash list, reflog, editor local history.

WHAT REALLY HAPPENS

Staged objects can exist without a convenient commit reference, but locating them is less direct and retention is limited. Start with known references and preserve evidence rather than running aggressive cleanup. Editor recovery and filesystem backup are independent mechanisms.

FOLLOW-UP QUESTION + ANSWER

Was the missing content ever added, committed or stashed?

That decides the recovery path: commits/reflog, stash, or potentially a staged blob may help. Never-saved working edits need editor or filesystem backups.

⚠ **INTERVIEW TRAP:** Reflog does not record every keystroke or every working-tree version.

Original question & example: uploaded Git guide, source page 29. Follow-up answer: added for this edition.

CHAPTER 10 / Reflog, recovery and clean-up Q 049 of 110

INTERVIEW QUESTION / Q049

How does git clean differ from reset --hard?

THE SHORT ANSWER

Clean removes selected untracked files; hard reset aligns index and tracked files with a commit. Clean has explicit options for directories and ignored paths and should be previewed.

COMMAND EXAMPLE / SCENARIO

```
git clean -nd          # preview untracked files/directories
   git clean -fd -- tmp-demo/  # delete only this intended path
   # -x also includes ignored paths; do not add it casually.
```

✎ REMEMBER git clean -nd # preview untracked files/directories

WHAT REALLY HAPPENS

Use a disposable lab or a clearly scoped path. Ignored files can contain local configuration, secrets or valuable generated output. Clean does not create a commit or a Git undo record for removed untracked content.

FOLLOW-UP QUESTION + ANSWER

What does the dry-run say will be deleted?

`git clean -nd` lists the untracked paths Git would remove if you ran the destructive form with matching flags. Inspect every listed path first.

⚠ **INTERVIEW TRAP:** git clean -fdx can remove ignored local files that the repository never backed up.

Original question & example: uploaded Git guide, source page 29. Follow-up answer: added for this edition.

CHAPTER 10 / Reflog, recovery and clean-up Q 050 of 110

INTERVIEW QUESTION / Q050

What are ORIG_HEAD and repository integrity checks useful for?

THE SHORT ANSWER

ORIG_HEAD can retain a previous tip for certain operations; integrity tools inspect objects and references. They help diagnosis but are not universal recovery guarantees.

COMMAND EXAMPLE / SCENARIO

```
git show ORIG_HEAD
   git fsck --full
   git count-objects -vH
```

✎ REMEMBER git show ORIG_HEAD

WHAT REALLY HAPPENS

ORIG_HEAD may be replaced by a later operation, so verify rather than assuming it refers to a particular incident. Fsck can report unreachable objects without those necessarily being corruption. Garbage collection and retention determine how long unreachable content survives.

FOLLOW-UP QUESTION + ANSWER

Is the repository corrupt or merely holding unreachable old commits?

`git fsck` output must be interpreted: unreachable valid commits are common after resets/rebases. Object corruption needs different evidence and repair steps.

⚠ **INTERVIEW TRAP:** Do not run aggressive pruning while trying to rescue unreachable work.

Original question & example: uploaded Git guide, source page 29. Follow-up answer: added for this edition.

CHAPTER 11 / Remotes, fetch and pull Q 051 of 110

INTERVIEW QUESTION / Q051

What are origin, upstream and origin/main?

THE SHORT ANSWER

> Origin is a conventional remote name created by clone. Upstream can be another remote name or the tracking branch relationship. Origin/main is normally a local remote-tracking reference.

COMMAND EXAMPLE / SCENARIO

```
git remote -v
   git branch -vv
   git rev-parse refs/remotes/origin/main
```

DRAW IT TO REMEMBER



Fetch inspects incoming history without integrating.

WHAT REALLY HAPPENS

Remote names are configurable. The remote-tracking reference is not a live query to the server, and your local main can differ from it. Clarify whether upstream means a named remote in a fork workflow or the branch used for comparison and default operations.

FOLLOW-UP QUESTION + ANSWER

What updates your local view of the remote branch?

`git fetch` updates your repository's configured remote-tracking references, such as `origin/main`, without merging them into your branch.

⚠ **INTERVIEW TRAP:** origin/main and local main are different references even when they currently have the same ID.

Original question & example: uploaded Git guide, source page 30. Follow-up answer: added for this edition.

CHAPTER 11 / Remotes, fetch and pull Q 052 of 110

INTERVIEW QUESTION / Q052

What does git fetch change?

THE SHORT ANSWER

Fetch downloads needed objects and updates references according to the refspec, normally remote-tracking branches. It does not ordinarily integrate those changes into your checked-out branch.

COMMAND EXAMPLE / SCENARIO

```
git fetch origin
   git log --oneline main..origin/main
   git diff main origin/main
```

✎ REMEMBER git fetch origin

WHAT REALLY HAPPENS

This lets you inspect incoming work before choosing merge or rebase. FETCH_HEAD also records fetched tips. Fetch configuration can limit which references update, and fetching tags has separate rules. Your files can remain unchanged while origin/main advances.

FOLLOW-UP QUESTION + ANSWER

How would you review incoming commits without integrating them?

Run `git fetch origin`, then `git log --oneline HEAD..origin/main` and `git diff HEAD..origin/main` before merging or rebasing.

⚠ **INTERVIEW TRAP:** A successful fetch is not the same as updating local main.

Original question & example: uploaded Git guide, source page 30. Follow-up answer: added for this edition.

CHAPTER 11 / Remotes, fetch and pull Q 053 of 110

INTERVIEW QUESTION / Q053

How does git pull differ from fetch?

THE SHORT ANSWER

Pull fetches and then integrates the selected upstream into the current branch using a merge, rebase or fast-forward-only policy. Make the policy explicit when learning or diagnosing divergence.

COMMAND EXAMPLE / SCENARIO

```
git pull --ff-only
   git pull --rebase
   git pull --no-rebase
```

✎ REMEMBER git pull --ff-only

WHAT REALLY HAPPENS

These are alternative policies. Ff-only refuses divergence; rebase replays local commits; no-rebase uses merge integration. Defaults depend on version and configuration, so inspect them rather than assuming pull always creates a merge.

FOLLOW-UP QUESTION + ANSWER

Which pull policy should reject accidental divergent local main commits?

`git pull --ff-only` refuses divergent histories; choose it when main must update only by fast-forward.

⚠ **INTERVIEW TRAP:** Pull can modify local history or working content after the fetch stage.

Original question & example: uploaded Git guide, source page 31. Follow-up answer: added for this edition.

CHAPTER 11 / Remotes, fetch and pull Q 054 of 110

INTERVIEW QUESTION / Q054

How do you set and inspect a branch's upstream?

THE SHORT ANSWER

An upstream links a local branch to a tracking target used by several default operations and comparisons. Set it deliberately and inspect branch configuration.

COMMAND EXAMPLE / SCENARIO

```
git branch --set-upstream-to=origin/main main
   git branch -vv
   git rev-parse --abbrev-ref '@{upstream}'
```

✎ REMEMBER git branch --set-upstream-to=origin/main main

WHAT REALLY HAPPENS

The target remote-tracking reference must exist. Upstream does not automatically push, grant permissions or make branch content identical. Push behavior also depends on push configuration and an explicitly provided refspec.

FOLLOW-UP QUESTION + ANSWER

Why can a branch be ahead even when it tracks origin/main?

Ahead is based on your local upstream-tracking reference, not a live server call. Your local commits can be ahead until you push or realign history.

⚠ **INTERVIEW TRAP:** Tracking configuration is not a background synchronization service.

Original question & example: uploaded Git guide, source page 31. Follow-up answer: added for this edition.

CHAPTER 11 / Remotes, fetch and pull Q 055 of 110

INTERVIEW QUESTION / Q055

What do fetch --prune and remote branch deletion mean?

THE SHORT ANSWER

Prune removes stale local tracking references covered by the fetch/prune mapping when their remote counterparts are gone. Deleting a remote branch is a separate server mutation.

COMMAND EXAMPLE / SCENARIO

```
git fetch --prune origin
   git branch -r
   git push origin --delete feature/finished
```

✎ REMEMBER git fetch --prune origin

WHAT REALLY HAPPENS

The deletion line requires the team's authorization and can be rejected by server policy. Prune does not normally delete your unrelated local feature branch. Tag pruning is sensitive to the configured refspec and explicit tag-prune options.

FOLLOW-UP QUESTION + ANSWER

Why can a local feature still exist after its remote branch is deleted?

Removing a remote branch deletes a server name. Your independent local `refs/heads/feature` remains until you delete it.

⚠ **INTERVIEW TRAP:** Do not assume pruning always affects only branches under every possible configuration.

Original question & example: uploaded Git guide, source page 31. Follow-up answer: added for this edition.

CHAPTER 12 / Push, rejected updates and leases Q 056 of 110

INTERVIEW QUESTION / Q056

What does git push actually do?

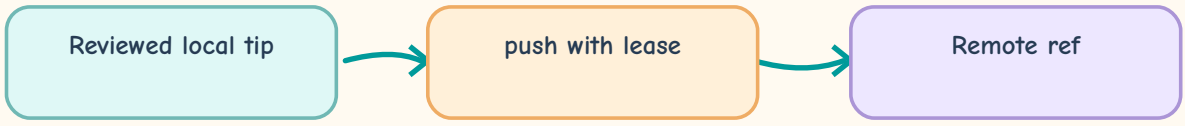
THE SHORT ANSWER

Push transfers required objects and asks the remote to update specified references. The requested update must satisfy Git's checks and the server's policy.

COMMAND EXAMPLE / SCENARIO

```
git push -u origin feature/orders
   git ls-remote --heads origin
```

DRAW IT TO REMEMBER



Lease helps detect an unexpected remote update.

WHAT REALLY HAPPENS

The -u option records an upstream for a successful or up-to-date branch push. It does not create review approval. Explicit remote and branch names reduce surprises from defaults, especially when fetch and push destinations differ.

FOLLOW-UP QUESTION + ANSWER

What does the server check before advancing a branch?

For a normal push, the server checks permissions, hooks/policy and whether the new tip fast-forwards the existing branch; exceptions require authorization.

⚠ **INTERVIEW TRAP:** A push can transfer objects yet still fail to update the requested reference.

Original question & example: uploaded Git guide, source page 32. Follow-up answer: added for this edition.

CHAPTER 12 / Push, rejected updates and leases Q 057 of 110

INTERVIEW QUESTION / Q057

Why is a non-fast-forward push rejected?

THE SHORT ANSWER

> For a normal branch update, the old remote tip should be an ancestor of the new tip. If it is not, updating would discard remote reachability from that branch and is normally rejected.

COMMAND EXAMPLE / SCENARIO

```
git fetch origin
   git log --left-right --oneline HEAD...origin/feature/orders
   # integrate or coordinate after inspecting both histories
```

✎ REMEMBER git fetch origin

WHAT REALLY HAPPENS

Another person may have pushed, or you may have rewritten local history. Fetching first reveals the actual divergence. Merge or rebase according to policy, test, and push the integrated result. A rejection is useful evidence, not a reason to force blindly.

FOLLOW-UP QUESTION + ANSWER

Are the remote-only commits someone else's work or your old rewritten series?

Fetch and graph the tips; compare `git log --left-right --cherry-pick` or `git range-diff` to learn whether remote commits are new work or rewritten equivalents.

⚠ **INTERVIEW TRAP:** Retrying the same ordinary push does not resolve divergent ancestry.

Original question & example: uploaded Git guide, source page 33. Follow-up answer: added for this edition.

CHAPTER 12 / Push, rejected updates and leases Q 058 of 110

INTERVIEW QUESTION / Q058

How does --force-with-lease differ from --force?

THE SHORT ANSWER

Force bypasses the usual ancestry protection. Force-with-lease also requires the remote reference to match an expected value, which can prevent overwriting a concurrent update.

COMMAND EXAMPLE / SCENARIO

```
# Authorized rewrite only; EXPECTED_OID is the reviewed old remote tip.
   git push --force-with-lease=refs/heads/feature:EXPECTED_OID origin HEAD:feature
```

✎ REMEMBER # Authorized rewrite only; EXPECTED_OID is the reviewed old remote tip.

WHAT REALLY HAPPENS

An explicit expected object ID makes the reviewed expectation clear. The shorthand form commonly relies on local remote-tracking state; background fetches can update that state and weaken your intended protection. A lease does not make rewriting shared history socially or semantically safe.

FOLLOW-UP QUESTION + ANSWER

What happens if a coworker pushes after you record EXPECTED_OID?

The explicit lease no longer matches, so the remote rejects the force update. Fetch, inspect the coworker's commit and coordinate a safe next step.

⚠ **INTERVIEW TRAP:** Force-with-lease can still replace work if your expected tip includes work you did not preserve.

Original question & example: uploaded Git guide, source page 33. Follow-up answer: added for this edition.

CHAPTER 12 / Push, rejected updates and leases Q 059 of 110

INTERVIEW QUESTION / Q059

How do push refspecs select the destination?

THE SHORT ANSWER

A push refspec maps a source object/reference to a destination reference. Explicit full destinations are useful for scripts where ambiguity is unacceptable.

COMMAND EXAMPLE / SCENARIO

```
git push origin HEAD:refs/heads/review/order-fix
   git push origin refs/tags/v1.0.0:refs/tags/v1.0.0
```

✎ **REMEMBER** git push origin HEAD:refs/heads/review/order-fix

WHAT REALLY HAPPENS

The first command publishes the current commit under a chosen server branch name, which can differ from the local branch. A leading plus requests a forced update and needs the same caution as force. Deletion has a dedicated command form.

FOLLOW-UP QUESTION + ANSWER

Can you push a detached HEAD to a named review branch?

Yes. Use an explicit refspec such as `git push origin HEAD:refs/heads/review/my-fix` if you are allowed to create that branch.

⚠ **INTERVIEW TRAP:** Choosing the wrong destination can publish to an unintended branch even with correct content.

Original question & example: uploaded Git guide, source page 34. Follow-up answer: added for this edition.

CHAPTER 12 / Push, rejected updates and leases Q 060 of 110

INTERVIEW QUESTION / Q060

How do you diagnose authentication, authorization and policy push failures?

THE SHORT ANSWER

Read the failure category, inspect the remote URL and intended ref, and check credentials versus server policy separately. Preserve local commits while diagnosing the network operation.

COMMAND EXAMPLE / SCENARIO

```
git remote get-url --push origin
   git branch --show-current
   git status
```

✎ REMEMBER git remote get-url --push origin

WHAT REALLY HAPPENS

Authentication proves an identity; authorization permits access; protected-branch or hook policy can reject an otherwise authenticated update. Do not expose tokens in diagnostic output. A local commit remains local even if its push fails.

FOLLOW-UP QUESTION + ANSWER

Does the failure occur before repository access or after reference-update validation?

Check the exact error: DNS/SSH/HTTPS failures happen before access, while 'non-fast-forward', hooks and protected-branch errors happen during update checks.

⚠ **INTERVIEW TRAP:** Changing user.email usually does not repair a transport permission error.

Original question & example: uploaded Git guide, source page 34. Follow-up answer: added for this edition.

CHAPTER 13 / Tags, releases and reproducible builds Q 061 of 110

INTERVIEW QUESTION / Q061

How do lightweight and annotated tags differ?

THE SHORT ANSWER

A lightweight tag is a reference to an object. An annotated tag has a tag object containing metadata and a message, and can also carry a signature.

COMMAND EXAMPLE / SCENARIO

```
git tag demo-light
   git tag -a v1.0.0 -m "Release 1.0.0"
   git cat-file -t v1.0.0
```

DRAW IT TO REMEMBER



Tags are release pointers; verify artifacts too.

WHAT REALLY HAPPENS

Both can identify a release commit, but annotated tags provide a separate tagger identity and message. A tag can point to objects other than commits, though release workflows commonly use commits. Normal commits do not move existing tags.

FOLLOW-UP QUESTION + ANSWER

What extra object exists for an annotated tag?

An annotated tag has its own `tag` object holding tagger, message and target. A lightweight tag is only a ref pointing directly to an object.

⚠ **INTERVIEW TRAP:** A tag name is a reference, so it can be changed unless policy prevents it.

Original question & example: uploaded Git guide, source page 35. Follow-up answer: added for this edition.

CHAPTER 13 / Tags, releases and reproducible builds Q 062 of 110

INTERVIEW QUESTION / Q062

Are tags pushed automatically with branches?

THE SHORT ANSWER

Do not assume they are. Push the selected release tag explicitly, or use a deliberate tag-publishing policy such as follow-tags for eligible annotated tags.

COMMAND EXAMPLE / SCENARIO

```
git push origin v1.0.0
   git push --follow-tags origin main
   git ls-remote --tags origin
```

✎ REMEMBER git push origin v1.0.0

WHAT REALLY HAPPENS

Follow-tags is not identical to pushing every local tag. An explicit tag push is easiest to review for one release. Server policy may restrict tag creation or updates. Check the remote reference rather than assuming a successful branch push published the release name.

FOLLOW-UP QUESTION + ANSWER

Which exact tag should a release pipeline create and publish?

Use a reviewed, immutable release commit and a uniquely named annotated tag such as `v2.4.0`; publish via the approved release process.

⚠ INTERVIEW TRAP: git push --tags can publish unintended local tags as well.

Original question & example: uploaded Git guide, source page 35. Follow-up answer: added for this edition.

CHAPTER 13 / Tags, releases and reproducible builds Q 063 of 110

INTERVIEW QUESTION / Q063

Why should release tags usually be immutable?

THE SHORT ANSWER

Moving an existing release name makes the same name refer to different source across clones and time. New release or correction names preserve reproducibility and audit meaning.

COMMAND EXAMPLE / SCENARIO

```
Deployed artifact metadata:
   release = v1.0.0
   commit = full object ID
   artifact digest = verified build output identity
```

✎ REMEMBER Deployed artifact metadata:

WHAT REALLY HAPPENS

Git itself can permit some tag operations depending on flags and server policy, so immutability is a team rule to enforce. A source ID does not fully specify a build without dependency, toolchain and configuration information.

FOLLOW-UP QUESTION + ANSWER

How can you prove which commit produced a running service?

Embed the source commit OID and artifact digest/build ID in release metadata; compare them to the deployed artifact and CI provenance.

⚠ **INTERVIEW TRAP:** Retagging v1.0.0 does not automatically fix artifacts already built from the old target.

Original question & example: uploaded Git guide, source page 36. Follow-up answer: added for this edition.

CHAPTER 13 / Tags, releases and reproducible builds Q 064 of 110

INTERVIEW QUESTION / Q064

What does git describe do?

THE SHORT ANSWER

Describe produces a human-readable name based on reachable tags and commit distance, subject to its options and available history. It can help diagnostics but is not a replacement for the full commit ID.

COMMAND EXAMPLE / SCENARIO

```
git describe --tags --always
   git rev-parse HEAD
   git status --porcelain
```

✎ REMEMBER git describe --tags --always

WHAT REALLY HAPPENS

A shallow clone or missing tags can produce a different description. The dirty option can signal working-tree modifications, but build reproducibility requires more than a label. Save exact source and artifact identities in build metadata.

FOLLOW-UP QUESTION + ANSWER

Why can local and CI descriptions differ for the same checked-out commit?

`git describe` depends on which tags are present/fetched and the history graph. CI clones may be shallow or may not fetch all tags.

⚠ **INTERVIEW TRAP:** A description may change when additional reachable tags are fetched.

Original question & example: uploaded Git guide, source page 36. Follow-up answer: added for this edition.

CHAPTER 13 / Tags, releases and reproducible builds Q 065 of 110

INTERVIEW QUESTION / Q065

How should a Java release branch receive a hotfix?

THE SHORT ANSWER

Backport the smallest coherent fix, test it against the release's dependencies and JDK, and publish a new version under the release policy. Record the relationship to the original fix.

COMMAND EXAMPLE / SCENARIO

```
git switch release/1.4
   git cherry-pick -x FIX_COMMIT
   ./mvnw verify
   git tag -a v1.4.1 -m "Fix quantity validation"
```

✎ REMEMBER git switch release/1.4

WHAT REALLY HAPPENS

Also ensure the required fix exists on future development lines so it is not lost in the next release. Database migrations and feature flags can make rollback more complex than selecting an old Git tag. A new source tag should correspond to a tested artifact.

FOLLOW-UP QUESTION + ANSWER

Does the fix depend on a schema change unavailable in production?

Check the current production schema and code dependencies; adapt or stage the migration safely before backporting a fix that relies on them.

⚠ **INTERVIEW TRAP:** A hotfix that compiles on main may fail on an older release toolchain.

Original question & example: uploaded Git guide, source page 37. Follow-up answer: added for this edition.

CHAPTER 14 / Stash and multiple worktrees Q 066 of 110

INTERVIEW QUESTION / Q066

What does git stash save by default?

THE SHORT ANSWER

A normal stash records tracked working-tree and index changes, then cleans those changes from the checkout. Untracked and ignored files require additional options.

COMMAND EXAMPLE / SCENARIO

```
git stash push -m "unfinished order validation"
   git stash list
   git stash show -p 'stash@{0}'
```

DRAW IT TO REMEMBER



Stash and worktrees serve different task changes.

WHAT REALLY HAPPENS

Use -u to include untracked files and -a when deliberately including ignored files too. Ignored paths may contain large output or local secrets, so choose consciously. A stash is stored locally and is not sent by an ordinary branch push.

FOLLOW-UP QUESTION + ANSWER

Which option includes the new file without including ignored build output?

Use `git stash push -u` to include untracked files; ignored outputs remain excluded unless you deliberately use `-a`.

⚠ **INTERVIEW TRAP:** A new untracked Java file is not saved by the default stash.

Original question & example: uploaded Git guide, source page 38. Follow-up answer: added for this edition.

CHAPTER 14 / Stash and multiple worktrees Q 067 of 110

INTERVIEW QUESTION / Q067

How do stash apply and stash pop differ?

THE SHORT ANSWER

Apply restores a stash without dropping it. Pop restores and removes it on success; if conflicts prevent a clean application, the stash normally remains for resolution.

COMMAND EXAMPLE / SCENARIO

```
git stash apply 'stash@{0}'
   git status
   # after verifying, deliberately drop the intended entry:
   git stash drop 'stash@{0}'
```

✎ REMEMBER git stash apply 'stash@{0}'

WHAT REALLY HAPPENS

Apply followed by verification is useful when preserving the saved state matters. Use --index when attempting to restore the staged state as well, understanding that conflicts can prevent it. Stash numbers can change after list operations.

FOLLOW-UP QUESTION + ANSWER

What should you check before dropping a stash after resolving conflicts?

Confirm your intended edits exist and work, `git status` has no unresolved conflicts, and nothing still needed remains only inside that stash.

⚠ **INTERVIEW TRAP:** Pop does not guarantee a conflict-free restoration on a changed base.

Original question & example: uploaded Git guide, source page 38. Follow-up answer: added for this edition.

CHAPTER 14 / Stash and multiple worktrees Q 068 of 110

INTERVIEW QUESTION / Q068

When is stash branch useful?

THE SHORT ANSWER

It creates a branch from the stash's original base and applies the saved work there, which can reduce conflicts caused by the current branch having moved far away.

COMMAND EXAMPLE / SCENARIO

```
git stash branch recover/order-work 'stash@{0}'
   git status
   git log --oneline -3
```

✎ REMEMBER git stash branch recover/order-work 'stash@{0}'

WHAT REALLY HAPPENS

When successfully using a stash reference, the applied stash entry is dropped according to the command's semantics. Review the recovered work and commit it deliberately. This is not the same as applying the stash on today's main.

FOLLOW-UP QUESTION + ANSWER

Why can the original base be easier to restore onto?

`git stash branch` checks out the original base where the stash was created, which may avoid conflicts caused by applying on a newer branch.

⚠ INTERVIEW TRAP: Do not assume the resulting branch includes all later main commits.

Original question & example: uploaded Git guide, source page 39. Follow-up answer: added for this edition.

CHAPTER 14 / Stash and multiple worktrees Q 069 of 110

INTERVIEW QUESTION / Q069

What is git worktree useful for?

THE SHORT ANSWER

Worktree lets one repository have multiple working directories with separate checked-out positions and indexes while sharing objects and most references. It supports parallel tasks without constant stashing.

COMMAND EXAMPLE / SCENARIO

```
git worktree add -b hotfix/order ../order-hotfix main
   git worktree list
   git -C ../order-hotfix status
```

✎ REMEMBER git worktree add -b hotfix/order ../order-hotfix main

WHAT REALLY HAPPENS

You can fix a release while keeping an unfinished feature checkout intact. Build output and environment files belong to each working directory, but common refs still affect the shared repository. Git normally prevents checking out the same branch in two worktrees.

FOLLOW-UP QUESTION + ANSWER

What state is private to a worktree and what is shared?

Working files, index and HEAD checkout are per worktree; repository objects and most refs/branches are shared. Git restricts one branch being checked out twice.

⚠ INTERVIEW TRAP: A worktree is not an independent clone with its own separate object database.

Original question & example: uploaded Git guide, source page 39. Follow-up answer: added for this edition.

CHAPTER 14 / Stash and multiple worktrees Q 070 of 110

INTERVIEW QUESTION / Q070

How do you remove or repair worktree metadata?

THE SHORT ANSWER

Use worktree-aware commands so Git can check state and update administrative records. Inspect status before removing a directory with useful work.

COMMAND EXAMPLE / SCENARIO

```
git -C ../order-hotfix status
   git worktree remove ../order-hotfix
   git worktree prune --dry-run
```

✎ REMEMBER git -C ../order-hotfix status

WHAT REALLY HAPPENS

Remove normally refuses problematic working state unless explicitly forced. Prune cleans stale administrative entries for missing worktrees; it is not the same as deleting a branch. Moving directories manually may require repair. Keep external or removable worktrees locked when appropriate.

FOLLOW-UP QUESTION + ANSWER

Why might a branch still appear checked out after its directory was removed?

A worktree can have stale administrative records after manual deletion. Inspect `git worktree list --porcelain` and prune only confirmed stale entries.

⚠ INTERVIEW TRAP: Deleting a worktree directory manually can leave stale records and lose uncommitted files.

Original question & example: uploaded Git guide, source page 40. Follow-up answer: added for this edition.

CHAPTER 15 / Ignore rules, attributes and cross-platform files Q 071 of 110

INTERVIEW QUESTION / Q071

Does .gitignore stop tracking an already committed file?

THE SHORT ANSWER

No. Ignore rules apply to untracked discovery; an already tracked file remains tracked. To stop future tracking while keeping the local file, deliberately remove it from the index.

COMMAND EXAMPLE / SCENARIO

```
# after deciding this file should no longer be tracked:
   git rm --cached -- local.properties
   # add the ignore rule, then commit the tracking change
```

DRAW IT TO REMEMBER



Already tracked files need separate untracking.

WHAT REALLY HAPPENS

The removal becomes part of the next commit, while the file stays locally when the command succeeds. Other collaborators may lose their checked-out copy when adopting that removal, so communicate. Existing history still contains previous committed content.

FOLLOW-UP QUESTION + ANSWER

What additional action is needed if the file contained an active credential?

Rotate or revoke the credential immediately. Removing the file from a commit does not disable a secret already exposed in history.

⚠ **INTERVIEW TRAP:** Adding a secret file to .gitignore does not remove the secret from history.

Original question & example: uploaded Git guide, source page 41. Follow-up answer: added for this edition.

CHAPTER 15 / Ignore rules, attributes and cross-platform files Q 072 of 110

INTERVIEW QUESTION / Q072

How do ignore patterns and negation work?

THE SHORT ANSWER

Patterns match paths within their scope; a leading exclamation can re-include a match. You cannot re-include a file inside an excluded directory without also making the relevant directory traversable.

COMMAND EXAMPLE / SCENARIO

```
target/
   *.log
   config/*
   !config/example.yml
```

✎ REMEMBER target/

WHAT REALLY HAPPENS

This keeps an example under a traversable config directory while excluding its siblings. Root anchoring, directory slashes and nested ignore files change matching. Use check-ignore to identify the rule rather than guessing.

FOLLOW-UP QUESTION + ANSWER

Which rule explains why a new file is ignored?

Use `git check-ignore -v PATH` to reveal the matching ignore rule and its source file.

⚠ **INTERVIEW TRAP:** config/ followed only by !config/example.yml will not behave like config/* in this example.

Original question & example: uploaded Git guide, source page 41. Follow-up answer: added for this edition.

CHAPTER 15 / Ignore rules, attributes and cross-platform files Q 073 of 110

INTERVIEW QUESTION / Q073

Which files should a Java project commonly ignore?

THE SHORT ANSWER

Ignore generated build outputs and machine-specific local state, while tracking source, tests, intended build configuration and the wrapper files required by the project's build policy.

COMMAND EXAMPLE / SCENARIO

```
Typical ignored output: target/, build/, *.class.
   Track by policy: pom.xml, build.gradle, wrapper scripts/config.
   Keep real credentials outside committed local configuration.
```

✎ REMEMBER Typical ignored output: target/, build/, *.class.

WHAT REALLY HAPPENS

Do not blindly ignore every .jar: some build-wrapper layouts or deliberately vendored dependencies need tracked binary files. Check the specific wrapper version and repository policy. Share sanitized example configuration rather than committing a developer's actual secrets.

FOLLOW-UP QUESTION + ANSWER

Can a clean clone build without files from your laptop?

A clean clone must have all required tracked sources, build scripts and dependency access. Run CI from a fresh clone to catch machine-only files.

⚠ INTERVIEW TRAP: A broad ignore rule can hide a required new build file and break CI.

Original question & example: uploaded Git guide, source page 42. Follow-up answer: added for this edition.

CHAPTER 15 / Ignore rules, attributes and cross-platform files Q 074 of 110

INTERVIEW QUESTION / Q074

What does .gitattributes control?

THE SHORT ANSWER

Attributes define path-based behavior such as text normalization, diff/merge drivers, binary treatment and filters. They can make repository behavior more consistent across platforms.

COMMAND EXAMPLE / SCENARIO

```
* text=auto
   *.java text eol=lf
   *.sh text eol=lf
   *.png binary
```

✎ REMEMBER * text=auto

WHAT REALLY HAPPENS

Line-ending normalization affects index content and checkout behavior under the configured rules. Introduce renormalization as a separate reviewed change where needed, then test rather than mixing it with a functional patch. Filters need consistent installation and trust decisions.

FOLLOW-UP QUESTION + ANSWER

How would you separate normalization from a production bug fix?

Make a separate commit for line-ending/format changes and another for the bug fix; confirm `.gitattributes` and compare with whitespace-insensitive tools.

⚠ **INTERVIEW TRAP:** Changing attributes can create many apparent diffs without changing application logic.

Original question & example: uploaded Git guide, source page 42. Follow-up answer: added for this edition.

CHAPTER 15 / Ignore rules, attributes and cross-platform files Q 075 of 110

INTERVIEW QUESTION / Q075

How do you diagnose unexpected ignored files or line-ending diffs?

THE SHORT ANSWER

Inspect the matching ignore rule, effective attributes and relevant configuration. Distinguish an intentional content change from normalization or executable-mode differences.

COMMAND EXAMPLE / SCENARIO

```
git check-ignore -v -- config/local.yml
   git check-attr text eol -- src/Order.java
   git diff --check
   git config --show-origin --get core.autocrlf
```

✎ REMEMBER git check-ignore -v -- config/local.yml

WHAT REALLY HAPPENS

Check-ignore normally focuses on untracked paths; tracked-path investigation may need its no-index option. Diff --check flags certain whitespace errors but does not prove semantic equality. Investigate the exact patch before applying a global setting.

FOLLOW-UP QUESTION + ANSWER

Why does a file appear wholly changed after a Windows checkout?

CRLF/LF conversion can change nearly every line. Inspect `git diff --ignore-space-at-eol` and repository `.gitattributes` before assuming logic changed.

⚠ INTERVIEW TRAP: Turning off all whitespace checks can hide real formatting problems without fixing normalization.

Original question & example: uploaded Git guide, source page 43. Follow-up answer: added for this edition.

CHAPTER 16 / Git objects, references and the index Q 076 of 110

INTERVIEW QUESTION / Q076

What are blob, tree, commit and tag objects?

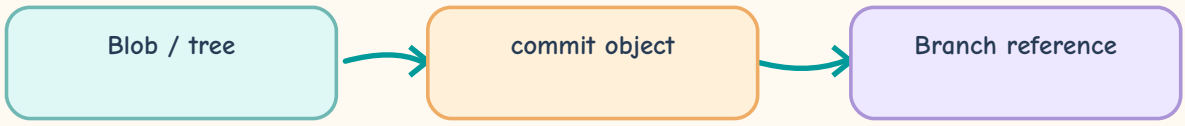
THE SHORT ANSWER

A blob stores file content; a tree maps names and modes to objects; a commit points to a tree and parents with metadata; an annotated tag adds named-object metadata.

COMMAND EXAMPLE / SCENARIO

```
git cat-file -t HEAD
   git cat-file -p HEAD
   git ls-tree HEAD
```

DRAW IT TO REMEMBER



Content objects are identified by their data.

WHAT REALLY HAPPENS

File names belong to tree entries, not the blob itself, so identical content can be reused at different paths. Trees also represent subtrees and special entry modes. The branch name is a reference outside the commit object.

FOLLOW-UP QUESTION + ANSWER

Why can two files with identical bytes share a blob ID?

Blob IDs hash file content, not names or paths. Identical bytes therefore reference the same blob object when using the same object format.

⚠ **INTERVIEW TRAP:** A blob does not inherently know which Java filename contained its content.

Original question & example: uploaded Git guide, source page 44. Follow-up answer: added for this edition.

CHAPTER 16 / Git objects, references and the index Q 077 of 110

INTERVIEW QUESTION / Q077

Why do references matter for object reachability?

THE SHORT ANSWER

References name starting points from which commit/tree/object relationships make content reachable. Removing a branch name does not immediately delete every object formerly reachable through it.

COMMAND EXAMPLE / SCENARIO

```
git show-ref --heads --tags
   git for-each-ref --format='%(refname) %(objectname)'
   git rev-list --all
```

✎ REMEMBER git show-ref --heads --tags

WHAT REALLY HAPPENS

Another branch, tag or reflog may still protect the commits. Eventually unreachable objects may be pruned according to retention and maintenance rules. Preserve a rescue reference before relying on uncertain retention.

FOLLOW-UP QUESTION + ANSWER

Which other references could still retain a deleted feature's commits?

Tags, remote-tracking refs, other branches, stash and reflog entries may still make commits reachable after deleting one branch name.

⚠ **INTERVIEW TRAP:** Deleting a branch is not a secure content-erasure operation.

Original question & example: uploaded Git guide, source page 44. Follow-up answer: added for this edition.

CHAPTER 16 / Git objects, references and the index Q 078 of 110

INTERVIEW QUESTION / Q078

What is the index during a conflict?

THE SHORT ANSWER

In ordinary state the index holds one proposed entry per tracked path. During an unresolved merge it can hold base, ours and theirs entries in separate stages.

COMMAND EXAMPLE / SCENARIO

```
git ls-files -u
   git show :1:src/Order.java
   git show :2:src/Order.java
   git show :3:src/Order.java
```

✎ REMEMBER git ls-files -u

WHAT REALLY HAPPENS

Stage 1 is the base, 2 ours, and 3 theirs when those entries exist. A stage can be missing in add/delete conflicts. Staging a resolved file replaces the unmerged entries with the chosen normal entry. Operation-specific ours/theirs meaning still matters.

FOLLOW-UP QUESTION + ANSWER

What changes in the index after git add marks a path resolved?

Git replaces the path's conflict-stage entries (1/2/3) with a normal stage-0 index entry containing your resolved content.

⚠ **INTERVIEW TRAP:** Not every conflict has all three file versions available.

Original question & example: uploaded Git guide, source page 45. Follow-up answer: added for this edition.

CHAPTER 16 / Git objects, references and the index Q 079 of 110

INTERVIEW QUESTION / Q079

Does Git track renames directly?

THE SHORT ANSWER

Git stores snapshots and can infer renames when comparing them using similarity detection. A move command records the path/content changes; there is no universal permanent rename object.

COMMAND EXAMPLE / SCENARIO

```
git mv src/OldOrder.java src/Order.java
   git diff --staged --find-renames
   git log --follow -- src/Order.java
```

✎ REMEMBER git mv src/OldOrder.java src/Order.java

WHAT REALLY HAPPENS

Rename detection depends on comparison context, thresholds and limits. Large content edits can make a move harder to recognize. Follow is useful for a single path's history but should not be treated as an infallible record of identity.

FOLLOW-UP QUESTION + ANSWER

Why might Git show a delete and add instead of a rename?

Rename detection is a similarity-based presentation of delete/add changes. Major edits or thresholds may prevent Git from recognizing a rename.

⚠ **INTERVIEW TRAP:** A rename-only commit and a rename-plus-rewrite can be displayed differently.

Original question & example: uploaded Git guide, source page 45. Follow-up answer: added for this edition.

CHAPTER 16 / Git objects, references and the index Q 080 of 110

INTERVIEW QUESTION / Q080

What do packfiles and garbage collection do?

THE SHORT ANSWER

Packfiles compress and organize objects, potentially using delta representations. Maintenance can consolidate storage and eventually remove unreachable objects under retention rules.

COMMAND EXAMPLE / SCENARIO

```
git count-objects -vH
   git gc  # normal maintenance; not a recovery step
```

✎ REMEMBER git count-objects -vH

WHAT REALLY HAPPENS

The logical snapshot model remains the same even when objects are packed. Loose-object count alone is not total repository size. During recovery, avoid aggressive pruning that removes objects you are trying to rescue. Large historical binaries can keep storage large even after deletion from today's tree.

FOLLOW-UP QUESTION + ANSWER

Why can deleting a large file leave the repository large?

The old large blob remains reachable through historical commits. Deleting the current path does not rewrite prior snapshots or automatically shrink history.

⚠ **INTERVIEW TRAP:** Garbage collection is not a substitute for intentional history cleanup or backup.

Original question & example: uploaded Git guide, source page 46. Follow-up answer: added for this edition.

CHAPTER 17 / Debugging with bisect, blame and history search Q 081 of 110

INTERVIEW QUESTION / Q081

How does git bisect find a regression?

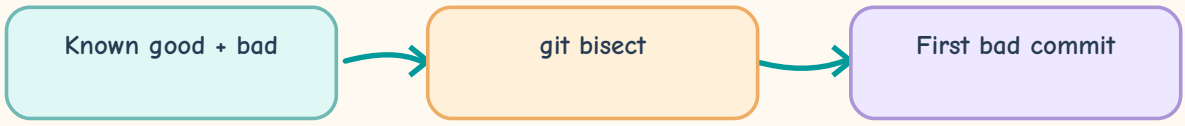
THE SHORT ANSWER

Bisect narrows a known good/bad history interval by checking intermediate commits and classifying their behavior. It is efficient when the test and good/bad boundary are reliable.

COMMAND EXAMPLE / SCENARIO

```
git bisect start
   git bisect bad BAD_TIP
   git bisect good GOOD_TIP
   # test each selected revision, then mark good, bad or skip
   git bisect reset
```

DRAW IT TO REMEMBER



Automate a reliable pass/fail test.

WHAT REALLY HAPPENS

The selected checkout may be detached while testing. Preserve local edits first and reset bisect when done. Flaky tests, environmental changes or a regression that disappears and returns can invalidate a simple monotonic interpretation.

FOLLOW-UP QUESTION + ANSWER

What evidence establishes the last known good revision?

Use a known passing CI build, release tag or tested deployment commit, then confirm it still passes the same deterministic reproduction check.

⚠ **INTERVIEW TRAP:** The newest failing commit is not necessarily the commit that introduced the problem.

Original question & example: uploaded Git guide, source page 47. Follow-up answer: added for this edition.

CHAPTER 17 / Debugging with bisect, blame and history search Q 082 of 110

INTERVIEW QUESTION / Q082

How does automated bisect use exit codes?

THE SHORT ANSWER

Bisect run executes a supplied test command. Zero means good; ordinary failure codes indicate bad, while 125 means the selected revision cannot be tested and should be skipped.

COMMAND EXAMPLE / SCENARIO

```
git bisect run ./regression-check.sh
   # test script contract:
   # 0 = regression absent; 1 = regression present; 125 = untestable
```

✎ REMEMBER git bisect run ./regression-check.sh

WHAT REALLY HAPPENS

Certain special exit codes stop the run, so use the documented contract. A Java test wrapper must distinguish the target regression from missing dependencies, incompatible JDKs or unrelated compilation failures. Otherwise it may blame an infrastructure break.

FOLLOW-UP QUESTION + ANSWER

How will the script classify an old commit requiring a different toolchain?

Return 'skip' (exit 125) for commits that cannot be tested fairly, such as those lacking required tools. Bad test setup must not count as product failure.

⚠ **INTERVIEW TRAP:** Mapping every build failure to bad can identify the wrong change.

Original question & example: uploaded Git guide, source page 48. Follow-up answer: added for this edition.

CHAPTER 17 / Debugging with bisect, blame and history search Q 083 of 110

INTERVIEW QUESTION / Q083

What does git blame tell you and what are its limits?

THE SHORT ANSWER

Blame attributes current lines to commits that last changed them under its traversal rules. It provides context for investigation, not proof of who introduced a behavioral bug.

COMMAND EXAMPLE / SCENARIO

```
git blame -L 20,50 -- src/OrderService.java
   git blame -w -- src/OrderService.java
   git show COMMIT_FROM_BLAME
```

✎ REMEMBER git blame -L 20,50 -- src/OrderService.java

WHAT REALLY HAPPENS

Formatting, moves and earlier dependencies can make the last line edit unrelated to the root cause. Inspect the patch, parent version and tests. Options can ignore whitespace or selected mechanical revisions when configured deliberately.

FOLLOW-UP QUESTION + ANSWER

Could the bug come from a caller that did not modify these lines?

Yes. `blame` identifies the last change to a line, not the full cause. Investigate callers, config, data and earlier design decisions.

⚠ **INTERVIEW TRAP:** The author of the last formatting commit may not own the behavior being investigated.

Original question & example: uploaded Git guide, source page 48. Follow-up answer: added for this edition.

CHAPTER 17 / Debugging with bisect, blame and history search Q 084 of 110

INTERVIEW QUESTION / Q084

How do -S and -G history searches differ?

THE SHORT ANSWER

Pickaxe -S searches for changes in the occurrence count of a string; -G searches for added or removed patch lines matching a regular expression. They answer different questions.

COMMAND EXAMPLE / SCENARIO

```
git log -S 'validateQuantity' --oneline -- src/
   git log -G 'quantity.*<.*0' -p -- src/
```

✎ REMEMBER git log -S 'validateQuantity' --oneline -- src/

WHAT REALLY HAPPENS

A moved string with the same count may not match -S even though relevant patch lines changed. Combine the search with path scope and inspect candidate commits. Search deleted history through suitable revisions instead of looking only at current files.

FOLLOW-UP QUESTION + ANSWER

Which query helps find when a validation expression changed?

Use `git log -S'validationText' -- path` for a changed string count, or `git log -G'regex' -- path` for matching patch lines.

⚠ **INTERVIEW TRAP:** Neither command is equivalent to a simple search of today's source tree.

Original question & example: uploaded Git guide, source page 48. Follow-up answer: added for this edition.

CHAPTER 17 / Debugging with bisect, blame and history search Q 085 of 110

INTERVIEW QUESTION / Q085

How does git grep help during an investigation?

THE SHORT ANSWER

It searches tracked content in the selected working/index or tree context, making it useful for finding related code at a particular revision without switching checkouts.

COMMAND EXAMPLE / SCENARIO

```
git grep -n 'validateQuantity'
   git grep -n 'validateQuantity' v1.0.0 -- src/
   git log --oneline -- src/OrderService.java
```

✎ REMEMBER git grep -n 'validateQuantity'

WHAT REALLY HAPPENS

The tagged search can reveal an older caller or implementation while leaving working files untouched. Choose options deliberately if untracked or ignored files also matter. Pair search results with history rather than concluding from one occurrence.

FOLLOW-UP QUESTION + ANSWER

How would you locate all uses in an older release snapshot?

Use `git grep 'symbol' RELEASE_TAG -- '*.java'` or `git grep 'symbol' COMMIT` to search the old snapshot without switching the working tree.

⚠ **INTERVIEW TRAP:** Today's grep result does not prove the same code existed in the release under investigation.

Original question & example: uploaded Git guide, source page 49. Follow-up answer: added for this edition.

CHAPTER 18 / Large repositories, sparse checkout, submodules and LFS Q 086 of 110

INTERVIEW QUESTION / Q086

What is a shallow clone and what can it affect?

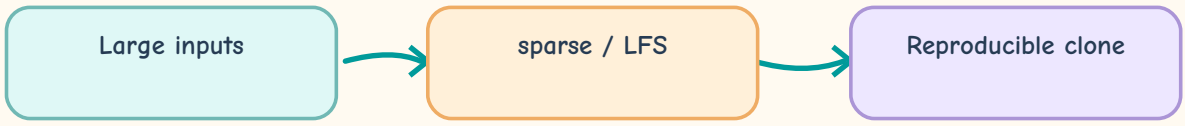
THE SHORT ANSWER

A shallow clone truncates commit ancestry at a chosen depth or boundary. It can speed acquisition but limits history-based commands until more history is fetched.

COMMAND EXAMPLE / SCENARIO

```
git clone --depth 20 REPOSITORY_URL demo
   git rev-parse --is-shallow-repository
   git fetch --unshallow
```

DRAW IT TO REMEMBER



Git pointers need reachable backing content.

WHAT REALLY HAPPENS

The final command requires a suitable remote with the missing history. Merge-base discovery, bisect, describe and release tooling may need deeper ancestry or tags. CI's actual checkout settings determine which data is present.

FOLLOW-UP QUESTION + ANSWER

Why might a pull-request diff fail to find the expected merge base?

A shallow or partial history may omit the common ancestor. Fetch/deepen the relevant history and verify refs before comparing.

⚠ **INTERVIEW TRAP:** A shallow clone is not merely a smaller working directory.

Original question & example: uploaded Git guide, source page 50. Follow-up answer: added for this edition.

CHAPTER 18 / Large repositories, sparse checkout, submodules and LFS Q 087 of 110

INTERVIEW QUESTION / Q087

How does a partial clone differ from a shallow clone?

THE SHORT ANSWER

A partial clone can omit selected objects, such as blobs, while keeping relevant commit structure and fetching promised objects when needed. Shallow clone limits ancestry.

COMMAND EXAMPLE / SCENARIO

```
git clone --filter=blob:none --no-checkout REPOSITORY_URL demo
   git -C demo sparse-checkout set service-a common
   git -C demo checkout
```

✎ REMEMBER git clone --filter=blob:none --no-checkout REPOSITORY_URL demo

WHAT REALLY HAPPENS

Server support and Git configuration matter. Reading missing content can trigger a network fetch, so offline behavior differs from a full clone. Combine partial clone with sparse checkout only when both serve the workload. These are illustrative commands, not a tested remote-server lab here.

FOLLOW-UP QUESTION + ANSWER

What happens when an offline command needs a missing blob?

In a partial clone, Git may attempt lazy fetching; offline access to an absent blob fails. Prefetch required objects before disconnecting.

⚠ **INTERVIEW TRAP:** Partial clone does not guarantee that later checkout avoids all network transfers.

Original question & example: uploaded Git guide, source page 50. Follow-up answer: added for this edition.

CHAPTER 18 / Large repositories, sparse checkout, submodules and LFS Q 088 of 110

INTERVIEW QUESTION / Q088

What does sparse checkout change?

THE SHORT ANSWER

Sparse checkout limits which tracked paths are populated in a working tree. It does not, by itself, erase the remaining paths from history or turn the repository into separate projects.

COMMAND EXAMPLE / SCENARIO

```
git sparse-checkout init --cone
   git sparse-checkout set service-a common
   git sparse-checkout list
   git sparse-checkout disable
```

✎ REMEMBER git sparse-checkout init --cone

WHAT REALLY HAPPENS

Cone mode expresses directory-oriented selections with its own inclusion rules. Build files and shared modules may still be needed. Sparse index support and tool compatibility depend on version and configuration. Choose paths from the actual build dependency graph.

FOLLOW-UP QUESTION + ANSWER

Which shared directories must a Java service build still include?

Include every build input: project modules, shared libraries, parent POMs, wrappers, scripts and resources. Sparse patterns must not hide required paths.

⚠ **INTERVIEW TRAP:** Hiding another module from the checkout does not remove its Git history.

Original question & example: uploaded Git guide, source page 51. Follow-up answer: added for this edition.

CHAPTER 18 / Large repositories, sparse checkout, submodules and LFS Q 089 of 110

INTERVIEW QUESTION / Q089

What is a submodule and how do you initialize it?

THE SHORT ANSWER

A submodule records a specific commit of another repository, with location metadata normally in .gitmodules. Updating the submodule checkout and updating the parent pointer are separate changes.

COMMAND EXAMPLE / SCENARIO

```
git clone --recurse-submodules REPOSITORY_URL demo
   git submodule update --init --recursive
   git submodule status
```

✎ REMEMBER git clone --recurse-submodules REPOSITORY_URL demo

WHAT REALLY HAPPENS

The parent stores a gitlink entry for the selected submodule commit, not a copy of its ordinary history. To adopt a new submodule revision, commit the pointer change in the parent. Ensure the referenced commit is available to collaborators before publishing that pointer.

FOLLOW-UP QUESTION + ANSWER

What happens if the parent points to a submodule commit never pushed?

The submodule checkout fails to obtain that commit from its remote, and CI cannot reproduce the build. Publish the submodule commit before the parent reference.

⚠ INTERVIEW TRAP: Cloning the parent without initialization may not provide the submodule's source files.

Original question & example: uploaded Git guide, source page 51. Follow-up answer: added for this edition.

CHAPTER 18 / Large repositories, sparse checkout, submodules and LFS Q 090 of 110

INTERVIEW QUESTION / Q090

What problem does Git LFS solve, and what are its limits?

THE SHORT ANSWER

Git LFS stores pointer files in Git while keeping selected large content in a separate LFS store. It helps binary workflows but adds tooling, transfer and retention dependencies.

COMMAND EXAMPLE / SCENARIO

```
# Requires Git LFS installation and a compatible remote:
   git lfs install
   git lfs track '*.psd'
   git add .gitattributes
```

✎ REMEMBER # Requires Git LFS installation and a compatible remote:

WHAT REALLY HAPPENS

Tracking a new pattern does not automatically migrate every old binary in history. Clones and CI need access to the LFS objects, not just Git pointers. Storage policy and cleanup require coordination; deletion of a current path may leave historical content retained.

FOLLOW-UP QUESTION + ANSWER

Can CI build if it has the Git commit but cannot download the LFS object?

No, not if the build needs its bytes. The Git pointer alone is insufficient; CI must authenticate and fetch the matching LFS object.

⚠ **INTERVIEW TRAP:** An LFS pointer is not the actual binary artifact.

Original question & example: uploaded Git guide, source page 52. Follow-up answer: added for this edition.

CHAPTER 19 / Team workflows, review and Java CI
Q 091 of 110

INTERVIEW QUESTION / Q091

How do trunk-based development and Gitflow-style workflows differ?

THE SHORT ANSWER

Trunk-based work emphasizes frequent integration and short-lived branches. Gitflow-style work uses additional long-lived development and release/support lines. Choose based on release needs and operational cost.

COMMAND EXAMPLE / SCENARIO

```
Frequent deployment: short feature branch -> reviewed main.
   Supported product versions: main plus deliberate release lines.
   Incomplete feature: controlled feature flag when appropriate.
```

DRAW IT TO REMEMBER



Test the exact source tree being delivered.

WHAT REALLY HAPPENS

Long-lived lines increase divergence and backport work; very short-lived lines require reliable tests and change isolation. Workflow names do not guarantee quality. Define who owns a release, how fixes propagate and which history can be rewritten.

FOLLOW-UP QUESTION + ANSWER

Does the product need several independently supported release versions?

If multiple versions need fixes independently, release branches and controlled backports may be worthwhile; one maintained version can use a simpler trunk flow.

⚠ **INTERVIEW TRAP:** Keeping a branch open for months increases integration risk even if each commit is small.

Original question & example: uploaded Git guide, source page 53. Follow-up answer: added for this edition.

CHAPTER 19 / Team workflows, review and Java CI Q 092 of 110

INTERVIEW QUESTION / Q092

What is the difference between a pull request and a merge?

THE SHORT ANSWER

A pull request is a hosting-platform review and integration workflow. Merge is a Git operation joining histories. A platform may integrate an approved request using merge, squash or rebase policies.

COMMAND EXAMPLE / SCENARIO

```
Review: inspect requirements, diff, tests and compatibility.
   Integration policy: merge commit, squash or rebase.
   After integration: delete short-lived branch by team policy.
```

✎ **REMEMBER** Review: inspect requirements, diff, tests and compatibility.

WHAT REALLY HAPPENS

Review can detect semantic mistakes Git cannot see. Approval, required checks and protected branches belong to hosting configuration. The reviewed diff must match the final integration result, especially after late conflict resolution or base changes.

FOLLOW-UP QUESTION + ANSWER

Which exact commit or combined result did CI validate?

Record the exact integration commit/tree (merge queue or final merge result), not only the feature tip. Test what will actually ship.

⚠ **INTERVIEW TRAP:** A green branch build does not always test the eventual merge result with current main.

Original question & example: uploaded Git guide, source page 53. Follow-up answer: added for this edition.

CHAPTER 19 / Team workflows, review and Java CI Q 093 of 110

INTERVIEW QUESTION / Q093

How should CI select and record source for a Java build?

THE SHORT ANSWER

Build an explicit commit or defined integration result, record its identity and run the intended toolchain and checks from a clean environment. Detached HEAD in CI can be intentional.

COMMAND EXAMPLE / SCENARIO

```
git rev-parse HEAD
   git status --porcelain
   ./mvnw verify
   # attach source ID, JDK/build-tool versions and artifact digest
```

✎ **REMEMBER** git rev-parse HEAD

WHAT REALLY HAPPENS

If tooling needs tags, history or submodules, configure checkout accordingly. A clean source tree does not fix unpinned dependencies or environment-specific behavior. Deployment should use the tested artifact rather than rebuilding differently later.

FOLLOW-UP QUESTION + ANSWER

How do you link the deployed artifact to the exact tested source?

Publish the commit SHA, dependency/build metadata and immutable artifact digest with the deployment; verify the artifact against CI records.

⚠ **INTERVIEW TRAP:** A mutable branch name can advance after the pipeline starts.

Original question & example: uploaded Git guide, source page 54. Follow-up answer: added for this edition.

CHAPTER 19 / Team workflows, review and Java CI Q 094 of 110

INTERVIEW QUESTION / Q094

How should Java teams handle generated files and schema migrations in Git?

THE SHORT ANSWER

Give generated artifacts an explicit tracking policy, keep semantic source review clear and treat deployed migrations as versioned operational history. Avoid mixing mechanical regeneration with unrelated logic changes.

COMMAND EXAMPLE / SCENARIO

```
Review separately: generated client regeneration and business change.
   Ignore ordinary target/build output.
   New DB correction: add a new migration under the migration policy.
```

✎ **REMEMBER** Review separately: generated client regeneration and business change.

WHAT REALLY HAPPENS

Some generated sources are deliberately committed for distribution, while others are reproducibly generated during build. Database rollback depends on the migration system and data, not just Git revert. Changing an already applied migration can invalidate checksums or history.

FOLLOW-UP QUESTION + ANSWER

Can a clean clone generate exactly the committed client code?

Run code generation in clean CI, verify reproducible output and compare generated files or expected checksums; no developer-only tool should be required.

⚠ **INTERVIEW TRAP:** Reverting Java source does not automatically undo a production database change.

Original question & example: uploaded Git guide, source page 54. Follow-up answer: added for this edition.

CHAPTER 19 / Team workflows, review and Java CI Q 095 of 110

INTERVIEW QUESTION / Q095

How do you make conflict resolution part of code review?

THE SHORT ANSWER

Treat the resolved result as a new decision to inspect and test. Compare it with both sides and confirm that important requirements still hold before accepting the integration.

COMMAND EXAMPLE / SCENARIO

```
git show --cc MERGE_COMMIT
   git diff MERGE_COMMIT^1 MERGE_COMMIT -- src/
   git diff MERGE_COMMIT^2 MERGE_COMMIT -- src/
```

✎ **REMEMBER** git show --cc MERGE_COMMIT

WHAT REALLY HAPPENS

Parent comparisons expose what the merge added relative to each line. Combined display has its own filtering semantics, so it should not be your only inspection. Ask the responsible reviewers to verify nontrivial resolutions and rerun integration tests.

FOLLOW-UP QUESTION + ANSWER

Which requirement could be lost by the chosen resolution?

Conflict choices can drop validation, authorization, error handling or transaction checks; add tests for both branches' intended requirements.

⚠ **INTERVIEW TRAP:** A conflict-free final index does not prove that both branches' behavior was preserved.

Original question & example: uploaded Git guide, source page 55. Follow-up answer: added for this edition.

CHAPTER 20 / Hooks, credentials and secure history Q 096 of 110

INTERVIEW QUESTION / Q096

What are Git hooks and are local hooks enough to enforce policy?

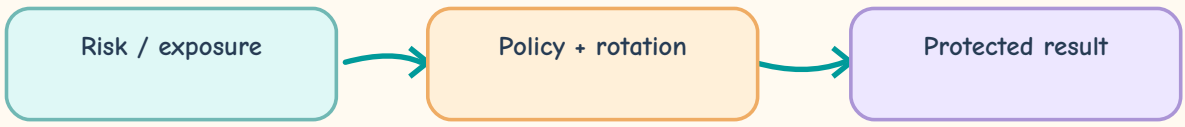
THE SHORT ANSWER

Hooks run at defined Git events. Local hooks help developers catch problems, but can be absent or bypassed. Critical shared policy needs suitable server or CI enforcement.

COMMAND EXAMPLE / SCENARIO

```
pre-commit: fast style or secret checks.
   commit-msg: message policy.
   pre-receive/update: server reference-update policy.
```

DRAW IT TO REMEMBER



Local hooks are not a server security boundary.

WHAT REALLY HAPPENS

Hook files are not automatically installed just because another person cloned the source repository. A configured hooks path can share reviewed scripts, but adoption and trust still matter. Avoid putting long unreliable checks in every local edit cycle.

FOLLOW-UP QUESTION + ANSWER

Where is the required security check enforced if a local hook is missing?

Server-side branch protection, required CI checks and review rules enforce policy independently of optional developer-side hooks.

⚠ **INTERVIEW TRAP:** --no-verify can bypass certain local hooks; it does not bypass server policy.

Original question & example: uploaded Git guide, source page 56. Follow-up answer: added for this edition.

CHAPTER 20 / Hooks, credentials and secure history Q 097 of 110

INTERVIEW QUESTION / Q097

How do HTTPS and SSH authentication compare?

THE SHORT ANSWER

Both can authenticate transport when configured correctly. HTTPS commonly uses a credential helper or token; SSH uses keys and host verification. Repository authorization remains a separate server decision.

COMMAND EXAMPLE / SCENARIO

```
git remote get-url origin
   git config --get credential.helper
   # use the organization's approved credential/key setup
```

✎ REMEMBER git remote get-url origin

WHAT REALLY HAPPENS

Choose the mechanism that fits the environment rather than claiming one is always more secure. Use least-privilege credentials, protect private keys and verify server identity. Avoid embedding tokens in URLs, shell history or committed configuration.

FOLLOW-UP QUESTION + ANSWER

Is the failing identity authorized for this specific repository and operation?

Verify the remote URL, selected SSH key/token identity, its scope and repository/branch rights; a credential can be valid but not authorized here.

⚠ **INTERVIEW TRAP:** Disabling host verification to fix a connection error removes an important trust check.

Original question & example: uploaded Git guide, source page 56. Follow-up answer: added for this edition.

CHAPTER 20 / Hooks, credentials and secure history Q 098 of 110

INTERVIEW QUESTION / Q098

What do signed commits and sign-off mean?

THE SHORT ANSWER

A cryptographic signature can support verification of commit provenance under a trusted key policy. A sign-off is a text trailer expressing a project's attestation convention; it is not the same as a signature.

COMMAND EXAMPLE / SCENARIO

```
git commit -S -m "Fix order validation"  # signing configured first
   git verify-commit SIGNED_COMMIT
   git commit -s -m "Fix order validation"  # adds sign-off trailer
```

✎ REMEMBER git commit -S -m "Fix order validation" # signing configured first

WHAT REALLY HAPPENS

Key trust, revocation and allowed signing identities determine what verification means. A signature does not prove the code is correct or approved. Rewriting a signed commit creates a different object that requires a new signature if the policy demands one.

FOLLOW-UP QUESTION + ANSWER

What should CI verify beyond the existence of a signature field?

Verify the cryptographic signature, trusted key identity, allowed signer policy and that protected build/ref policy used the verified commit.

⚠ **INTERVIEW TRAP:** A Signed-off-by line is not cryptographic proof of identity.

Original question & example: uploaded Git guide, source page 57. Follow-up answer: added for this edition.

CHAPTER 20 / Hooks, credentials and secure history Q 099 of 110

INTERVIEW QUESTION / Q099

What should you do if a secret was committed?

THE SHORT ANSWER

Revoke or rotate the exposed credential, assess its use, and coordinate removal from active files and history where required. Removing the latest file does not invalidate a copied credential.

COMMAND EXAMPLE / SCENARIO

```
Immediate action: revoke/rotate through the credential owner.
   Repository action: remove active secret; add safe example/config policy.
   History cleanup: coordinated rewrite with clones/forks and server handling.
```

✎ REMEMBER Immediate action: revoke/rotate through the credential owner.

WHAT REALLY HAPPENS

History rewrite is a separate operation with affected references, backups and collaborators. Use reviewed tooling and documented hosting guidance rather than assuming a force push erases every copy. The urgent guarantee is that the old credential stops working.

FOLLOW-UP QUESTION + ANSWER

How will you verify the exposed credential is no longer accepted?

Revoke/rotate it at the provider and prove the old credential fails a safe authentication test. Rewrite history only when warranted, with coordination.

⚠ **INTERVIEW TRAP:** .gitignore and git revert do not erase the credential from existing historical objects.

Original question & example: uploaded Git guide, source page 57. Follow-up answer: added for this edition.

CHAPTER 20 / Hooks, credentials and secure history Q 100 of 110

INTERVIEW QUESTION / Q100

Why does a repository need trust boundaries for automation?

THE SHORT ANSWER

A repository can contain build scripts, hooks configuration and workflow changes. Reviewing source or a pull request should not automatically grant untrusted code privileged execution with secrets.

COMMAND EXAMPLE / SCENARIO

```
External contribution: test in an appropriately limited environment.
   Trusted release: scoped credentials and protected approval path.
   Build output: verify provenance and source identity.
```

✎ REMEMBER External contribution: test in an appropriately limited environment.

WHAT REALLY HAPPENS

CI behavior depends on the platform's trigger and permission model. Keep secret-bearing jobs separated from untrusted execution as required by that model. Git ownership checks and server controls solve different trust problems.

FOLLOW-UP QUESTION + ANSWER

Which identity and secrets are available to the proposed pipeline run?

Review whether untrusted PR code receives any tokens, cloud roles, secrets or write permissions; use minimal scoped, short-lived permissions.

⚠ **INTERVIEW TRAP:** A change to a workflow file can be as sensitive as a change to production Java code.

Original question & example: uploaded Git guide, source page 58. Follow-up answer: added for this edition.

CHAPTER 21 / Advanced revisions, patches and repeated resolutions Q 101 of 110

INTERVIEW QUESTION / Q101

How do HEAD~2 and HEAD^2 differ?

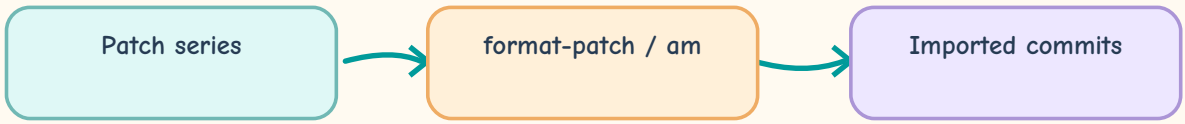
THE SHORT ANSWER

Tilde follows the first-parent chain a given number of steps. Caret selects a particular parent of one commit. HEAD^2 therefore requires a second parent, such as a merge commit.

COMMAND EXAMPLE / SCENARIO

```
git show HEAD~2
   git show HEAD^1
   git show HEAD^2  # only if HEAD has a second parent
```

DRAW IT TO REMEMBER



Always check parentage and dependencies.

WHAT REALLY HAPPENS

For ordinary one-parent history, HEAD~1 and HEAD^1 agree. At a merge, HEAD~2 follows parent 1 twice, while HEAD^2 selects the other parent immediately. Quoting revision expressions can be necessary in some shells.

FOLLOW-UP QUESTION + ANSWER

How would you inspect the feature tip recorded by a merge's second parent?

`git show MERGE_COMMIT^2` inspects the second parent's commit; `git log MERGE_COMMIT^2` follows its history.

⚠ **INTERVIEW TRAP:** ^2 does not mean two commits back.

Original question & example: uploaded Git guide, source page 59. Follow-up answer: added for this edition.

CHAPTER 21 / Advanced revisions, patches and repeated resolutions Q 102 of 110

INTERVIEW QUESTION / Q102

How do git apply and git am differ?

THE SHORT ANSWER

Apply applies a patch to files and optionally the index without creating the corresponding commit. Am imports suitable mail-format patches as commits with their metadata.

COMMAND EXAMPLE / SCENARIO

```
git apply --check fix.patch
   git apply fix.patch
   # separate workflow for format-patch output:
   git am 0001-fix-order-validation.patch
```

✎ REMEMBER git apply --check fix.patch

WHAT REALLY HAPPENS

Inspect patch origin and scope before applying. Apply --check checks applicability, not whether the code is safe or correct. Am has its own continue/abort flow when applying a series fails. Different patch formats preserve different metadata.

FOLLOW-UP QUESTION + ANSWER

Which workflow preserves the author and message from a format-patch series?

`git am` imports the mailbox patch series and typically preserves the supplied author and commit messages; check conflicts and results.

⚠ INTERVIEW TRAP: Applying a patch is not automatically committing it.

Original question & example: uploaded Git guide, source page 60. Follow-up answer: added for this edition.

CHAPTER 21 / Advanced revisions, patches and repeated resolutions Q 103 of 110

INTERVIEW QUESTION / Q103

What is format-patch used for?

THE SHORT ANSWER

It exports commits as mail-style patch files containing change content and commit information. It supports review or transfer workflows that do not depend on a hosting pull-request interface.

COMMAND EXAMPLE / SCENARIO

```
git format-patch -1 FIX_COMMIT
   git format-patch BASE_COMMIT..feature/orders
```

✎ **REMEMBER** git format-patch -1 FIX_COMMIT

WHAT REALLY HAPPENS

The receiver can inspect and apply the series with an appropriate am workflow. The resulting commit IDs may differ because destination ancestry and metadata differ. Patch transfer does not include every referenced object or large external artifact.

FOLLOW-UP QUESTION + ANSWER

What dependencies must be present for the series to apply and compile?

Ensure earlier patches, prerequisite types/APIs, and target dependencies are present. Apply in dependency order, then build and run tests.

⚠ **INTERVIEW TRAP:** A patch series is not a complete backup of all repository references.

Original question & example: uploaded Git guide, source page 60. Follow-up answer: added for this edition.

CHAPTER 21 / Advanced revisions, patches and repeated resolutions Q 104 of 110

INTERVIEW QUESTION / Q104

What is rerere and why should you still review its result?

THE SHORT ANSWER

Rerere records a conflict resolution and can reuse it when a matching conflict reappears. It saves repetitive work, but does not establish that an old resolution remains semantically correct.

COMMAND EXAMPLE / SCENARIO

```
git config --local rerere.enabled true
   git rerere status
   git diff
```

✎ REMEMBER git config --local rerere.enabled true

WHAT REALLY HAPPENS

Automatic staging is controlled separately. After reuse, inspect the resolved content and test it under the new surrounding code. The feature is useful for repeated integrations or rebases with similar conflicts. Remove an unsuitable recorded resolution deliberately rather than repeatedly accepting it.

FOLLOW-UP QUESTION + ANSWER

Should a reused resolution be staged automatically in this repository?

Usually inspect and stage the resolution deliberately. `rerere.autoupdate` can stage reused resolutions automatically, but team policy and tests should control it.

⚠ **INTERVIEW TRAP:** A syntactically matching conflict can have different business context later.

Original question & example: uploaded Git guide, source page 61. Follow-up answer: added for this edition.

CHAPTER 21 / Advanced revisions, patches and repeated resolutions Q 105 of 110

INTERVIEW QUESTION / Q105

How can merge-base and ancestry tests support automation?

THE SHORT ANSWER

Merge-base finds common ancestry, while an ancestor check answers whether one commit is reachable from another. They let scripts reason about topology instead of counting visible log lines.

COMMAND EXAMPLE / SCENARIO

```
git merge-base main feature/orders
   git merge-base --is-ancestor BASE_COMMIT HEAD
   # exit 0: ancestor; exit 1: not ancestor; errors are separate
```

✎ REMEMBER git merge-base main feature/orders

WHAT REALLY HAPPENS

Handle missing objects, shallow history and multiple merge bases under the script's requirements. A successful ancestry check says nothing about review, security or semantic equivalence. Squash-integrated patches can be present without original ancestry.

FOLLOW-UP QUESTION + ANSWER

Why can a squash-merged feature fail an ancestry check?

A squash merge copies combined changes into a new commit but not the original commits' parent links. An ancestry check therefore still reports them absent.

⚠ **INTERVIEW TRAP:** Equivalent content does not necessarily imply that the source commit is an ancestor.

Original question & example: uploaded Git guide, source page 61. Follow-up answer: added for this edition.

CHAPTER 22 / Senior interview scenarios and judgment Q 106 of 110

INTERVIEW QUESTION / Q106

You committed on the wrong branch: how do you move the work?

THE SHORT ANSWER

Preserve the commit under the intended branch, then decide how to repair the wrong branch according to whether it was shared. Avoid losing the only reference before creating the new one.

COMMAND EXAMPLE / SCENARIO

```
# if the accidental tip is private and should become a new feature:
   git switch -c feature/intended-work
   # inspect the old branch and deliberately restore its intended tip
```

DRAW IT TO REMEMBER



Explain which refs, files and people were affected.

WHAT REALLY HAPPENS

If the target branch already exists, cherry-pick the needed commits there instead. A private wrong branch may be reset after preserving work; a shared wrong branch normally needs an inverse or corrective commit. Verify uncommitted files as a separate concern.

FOLLOW-UP QUESTION + ANSWER

Was the accidental commit already published and used by teammates?

If published, protect teammates' history and normally add a correction/revert. If private, an amend/reset may be acceptable after saving work.

⚠ **INTERVIEW TRAP:** Deleting or resetting the original reference first can make recovery harder.

Original question & example: uploaded Git guide, source page 62. Follow-up answer: added for this edition.

CHAPTER 22 / Senior interview scenarios and judgment Q 107 of 110

INTERVIEW QUESTION / Q107

A teammate force-pushed and your branch diverged: what do you do?

THE SHORT ANSWER

Preserve your current tip, inspect old and new upstream histories, identify your unique work, and replay or integrate only what is intended. Coordinate before changing the server again.

COMMAND EXAMPLE / SCENARIO

```
git branch backup/local-before-repair
   git fetch origin
   git log --left-right --oneline HEAD...origin/main
```

✎ REMEMBER git branch backup/local-before-repair

WHAT REALLY HAPPENS

Blind pull can reintroduce commits the teammate deliberately replaced. The correct repair depends on the old base, equivalent patches and team agreement. Range-diff and explicit rebase --onto can help once boundaries are known. Test the reconstructed result.

FOLLOW-UP QUESTION + ANSWER

Which local commits represent your changes rather than the old upstream series?

Fetch the new upstream; inspect `git log --left-right --cherry-pick` and `git range-diff` against your backup to distinguish your patch series from upstream replacements.

⚠ **INTERVIEW TRAP:** Another force push can overwrite new valid work while trying to undo the first incident.

Original question & example: uploaded Git guide, source page 62. Follow-up answer: added for this edition.

CHAPTER 22 / Senior interview scenarios and judgment Q 108 of 110

INTERVIEW QUESTION / Q108

A merge succeeds but Java tests fail: why can that happen?

THE SHORT ANSWER

Git merges textual/tree changes, not business semantics. Independent edits can combine into invalid behavior even without overlapping lines, and clean integration still needs tests and review.

COMMAND EXAMPLE / SCENARIO

```
Branch A changes validation rules.
   Branch B changes the caller's assumptions.
   Merge has no text conflict, but checkout behavior regresses.
```

✎ REMEMBER Branch A changes validation rules.

WHAT REALLY HAPPENS

Inspect each parent's changes and the resulting interaction. Use a targeted regression test and consider whether the fix belongs in one branch or the integration layer. Avoid assuming that lack of conflict establishes application correctness.

FOLLOW-UP QUESTION + ANSWER

Which integration test covers the interaction between these two changes?

Create a test exercising both changes together, including the boundary where they interact (for example stock reservation and payment retry), not just separate unit tests.

⚠ **INTERVIEW TRAP:** Compiling alone can miss a behavioral or transaction-boundary regression.

Original question & example: uploaded Git guide, source page 63. Follow-up answer: added for this edition.

CHAPTER 22 / Senior interview scenarios and judgment Q 109 of 110

INTERVIEW QUESTION / Q109

Your local branch says ahead but pushing updates nothing: how do you investigate?

THE SHORT ANSWER

Inspect the exact upstream, push destination and refspec rather than assuming all defaults refer to the same server branch. Ahead is a comparison against a local tracking reference.

COMMAND EXAMPLE / SCENARIO

```
git branch -vv
   git remote -v
   git config --get push.default
   git remote get-url --push origin
```

✎ REMEMBER git branch -vv

WHAT REALLY HAPPENS

The tracking view may be stale or the configured push target may differ. Confirm whether the intended remote reference has the expected object ID. Do not expose credentials while printing configuration or URLs. An explicit reviewed source/destination can remove ambiguity.

FOLLOW-UP QUESTION + ANSWER

Which reference is the status comparison using?

The status compares against the branch's configured upstream tracking ref (shown by `git branch -vv`), which may differ from the push destination.

⚠ **INTERVIEW TRAP:** Ahead/behind is not a live measurement of every remote.

Original question & example: uploaded Git guide, source page 63. Follow-up answer: added for this edition.

CHAPTER 22 / Senior interview scenarios and judgment Q 110 of 110

INTERVIEW QUESTION / Q110

How should you answer an unfamiliar Git failure in an interview?

THE SHORT ANSWER

State what must be preserved, inspect status and topology, explain the probable category, and choose a reversible diagnostic before a destructive repair. Be precise about what you do not yet know.

COMMAND EXAMPLE / SCENARIO

```
Inspect: status, operation state, refs, diffs, recent reflog.
   Preserve: backup branch and valuable working files.
   Decide: integrate, undo, rescue or fix permissions/policy.
```

✎ REMEMBER Inspect: status, operation state, refs, diffs, recent reflog.

WHAT REALLY HAPPENS

Explain expected effects on HEAD, index, working tree and remote references. Confirm the outcome with an appropriate diff, graph and tests. A command is correct only relative to the desired end state and the collaboration boundary.

FOLLOW-UP QUESTION + ANSWER

What observable result would show that your repair succeeded?

Show the repaired state with a clean/reviewed graph, correct target ref and expected tests/CI. Confirm the intended remote OID if publication was part of the task.

⚠ **INTERVIEW TRAP:** A memorized reset --hard answer can destroy the very evidence needed to diagnose the problem.

Original question & example: uploaded Git guide, source page 64. Follow-up answer: added for this edition.

PRACTICE IN DISPOSABLE REPOSITORIES

14 real command labs

Run each lab in its **own newly created empty folder**, never inside a project with valuable files. All scripts perform precondition checks before modifying the directory.

L01–L04: staging, merge and rebase conflicts

L05–L08: reset, revert, reflog rescue and backports

L09–L11: remote leases, stash and worktrees

L12–L14: bisect, ignore rules and release tags

The Git guide originally tested these labs with Git 2.51.1. This edition independently retested them in isolated scratch folders using Git 2.47.3. The lab ZIP includes the scripts exactly as extracted, plus a README.

HANDS-ON GIT PRACTICE L01 / L14

TESTED BASH LAB • PASS

L01 Commit the staged version, not the later edit

Run from a **new empty directory**. Requires Bash and Git; works in Git Bash on Windows.

```
set -eu
# Run in a new, empty disposable directory.
test -z "$(ls -A)"
git init -q -b main
git config --local user.name "Git Learner"
git config --local user.email "learner@example.test"
git config --local commit.gpgsign false
git config --local core.autocrlf false
printf 'quantity=10\n' > Order.txt
git add Order.txt
git commit -qm "Initial quantity"
printf 'quantity=20\n' > Order.txt
git add Order.txt
printf 'quantity=30\n' > Order.txt
git commit -qm "Stage quantity 20"
test "$(git show HEAD:Order.txt)" = 'quantity=20'
test "$(cat Order.txt)" = 'quantity=30'
test -z "$(git diff --staged)"
test -n "$(git diff -- Order.txt)"
printf 'committed=20; working=30; unstaged diff remains\n'
```

VERIFIED OUTPUT

```
committed=20; working=30; unstaged diff remains
```

Why it works: git add recorded quantity 20 in the index. The later edit to 30 affected only the working tree. The normal commit used the index, then the staged diff became empty while the working diff remained.

CAUTION: A normal commit does not automatically include a later unstaged edit. Inspect both diff views before assuming all work was saved.

HANDS-ON GIT PRACTICE L02 / L14

TESTED BASH LAB • PASS

L02 Fast-forward versus an explicit merge commit

Run from a **new empty directory**. Requires Bash and Git; works in Git Bash on Windows.

```
set -eu
# Run in a new, empty disposable directory.
test -z "$(ls -A)"
git init -q -b main
git config --local user.name "Git Learner"
git config --local user.email "learner@example.test"
git config --local commit.gpgsign false
git config --local core.autocrlf false
printf 'base\n' > base.txt
git add base.txt
git commit -qm "Base"
git switch -qc feature
printf 'feature\n' > feature.txt
git add feature.txt
git commit -qm "Feature"
feature_tip=$(git rev-parse HEAD)
git switch -q main
git merge --ff-only feature > /dev/null
test "$(git rev-parse HEAD)" = "$feature_tip"
git switch -qc next-feature
printf 'next\n' > next.txt
git add next.txt
git commit -qm "Next feature"
git switch -q main
git merge --no-ff -m "Integrate next feature" next-feature > /dev/null
test "$(git rev-list --parents -n 1 HEAD | wc -w)" -eq 3
printf 'fast-forward reused tip; no-ff created a two-parent commit\n'
```

VERIFIED OUTPUT

```
fast-forward reused tip; no-ff created a two-parent commit
```

> **Why it works:** A fast-forward moves main to the existing feature commit. The second feature is also fast-forwardable, but --no-ff deliberately creates a commit whose parent list has two entries.

CAUTION: These history policies do not validate application behavior. Check the graph and tests as well as parent count.

HANDS-ON GIT PRACTICE L03 / L14

TESTED BASH LAB • PASS

L03 Abort and then resolve a real merge conflict

Run from a **new empty directory**. Requires Bash and Git; works in Git Bash on Windows.

```
set -eu
# Run in a new, empty disposable directory.
test -z "$(ls -A)"
git init -q -b main
git config --local user.name "Git Learner"
git config --local user.email "learner@example.test"
git config --local commit.gpgsign false
git config --local core.autocrlf false
printf 'price=100\n' > price.txt
git add price.txt
git commit -qm "Base price"
git switch -qc feature
printf 'price=90\n' > price.txt
git commit -qam "Feature price"
git switch -q main
printf 'price=110\n' > price.txt
git commit -qam "Main price"
if git merge feature > /dev/null 2>&1; then exit 1; fi
test -n "$(git ls-files -u)"
git merge --abort
test "$(cat price.txt)" = 'price=110'
test -z "$(git status --porcelain)"
if git merge feature > /dev/null 2>&1; then exit 1; fi
printf 'price=95\n' > price.txt
git add price.txt
git commit -qm "Combine price policy"
test -z "$(git ls-files -u)"
test "$(git show HEAD:price.txt)" = 'price=95'
test "$(git rev-list --parents -n 1 HEAD | wc -w)" -eq 3
printf 'abort restored 110; resolved merge records 95 and two parents\n'
```

VERIFIED OUTPUT

```
abort restored 110; resolved merge records 95 and two parents
```

> **Why it works:** Both branches change the same base line. The first merge stops with unmerged index entries; abort restores the clean pre-merge main state. The second merge is resolved to an explicit combined policy and committed.

CAUTION: Price 95 is a fixture decision, not a recommended business rule. In real code, verify the resolution with requirement-specific tests.

HANDS-ON GIT PRACTICE L04 / L14

TESTED BASH LAB • PASS

L04 Rebase conflict and operation-specific sides

Run from a **new empty directory**. Requires Bash and Git; works in Git Bash on Windows.

```
set -eu
# Run in a new, empty disposable directory.
test -z "$(ls -A)"
git init -q -b main
git config --local user.name "Git Learner"
git config --local user.email "learner@example.test"
git config --local commit.gpgsign false
git config --local core.autocrlf false
printf 'base\n' > policy.txt
git add policy.txt
git commit -qm "Base"
git switch -qc feature
printf 'feature\n' > policy.txt
git commit -qam "Feature policy"
old_tip=$(git rev-parse HEAD)
git branch backup/feature
git switch -q main
printf 'main\n' > policy.txt
git commit -qam "Main policy"
main_tip=$(git rev-parse HEAD)
git switch -q feature
if git rebase main > /dev/null 2>&1; then exit 1; fi
test "$(git show :2:policy.txt)" = 'main'
test "$(git show :3:policy.txt)" = 'feature'
printf 'combined\n' > policy.txt
git add policy.txt
GIT_EDITOR=true git rebase --continue > /dev/null 2>&1
test "$(git rev-parse HEAD^)" = "$main_tip"
test "$(git rev-parse HEAD)" != "$old_tip"
test "$(git rev-parse backup/feature)" = "$old_tip"
test "$(cat policy.txt)" = 'combined'
printf 'ours=main; theirs=feature; new tip has main as parent\n'
```

VERIFIED OUTPUT

```
ours=main; theirs=feature; new tip has main as parent
```

> **Why it works:** The merge backend treats the new base side as ours and the replayed feature as theirs. After resolution, the replayed commit has the new main tip as its parent and a different identity. The backup reference still names the original.

CAUTION: This does not authorize rewriting a shared branch. Inspect the full result and compare it to the preserved patch series before publishing.

HANDS-ON GIT PRACTICE L05 / L14

TESTED BASH LAB • PASS

L05 Measure soft, mixed and hard reset

Run from a **new empty directory**. Requires Bash and Git; works in Git Bash on Windows.

```
set -eu
# Run in a new, empty disposable directory.
test -z "$(ls -A)"
git init -q -b main
git config --local user.name "Git Learner"
git config --local user.email "learner@example.test"
git config --local commit.gpgsign false
git config --local core.autocrlf false
printf 'A\n' > value.txt
git add value.txt
git commit -qm "A"
a=$(git rev-parse HEAD)
printf 'B\n' > value.txt
git commit -qam "B"
git branch keep-B
git reset --soft "$a"
test "$(git show :value.txt)" = B
test "$(cat value.txt)" = B
test -n "$(git diff --staged)"
git reset --hard keep-B > /dev/null
git reset --mixed "$a" > /dev/null
test "$(git show :value.txt)" = A
test "$(cat value.txt)" = B
test -z "$(git diff --staged)"
test -n "$(git diff)"
git reset --hard keep-B > /dev/null
git reset --hard "$a" > /dev/null
test "$(cat value.txt)" = A
test "$(git rev-parse HEAD)" = "$a"
test -z "$(git status --porcelain)"
printf 'soft: B staged; mixed: B unstaged; hard: files A\n'
```

VERIFIED OUTPUT

```
soft: B staged; mixed: B unstaged; hard: files A
```

> **Why it works:** The target commit is A in every case. Soft keeps the B index and files; mixed restores the A index while retaining B files; hard restores A in both areas. keep-B deliberately preserves the original B commit throughout the exercise.

CAUTION: The hard resets here are confined to the disposable fixture. They can discard useful local edits in a real repository.

HANDS-ON GIT PRACTICE L06 / L14

TESTED BASH LAB • PASS

L06 Revert a change while preserving later history

Run from a **new empty directory**. Requires Bash and Git; works in Git Bash on Windows.

```
set -eu
# Run in a new, empty disposable directory.
test -z "$(ls -A)"
git init -q -b main
git config --local user.name "Git Learner"
git config --local user.email "learner@example.test"
git config --local commit.gpgsign false
git config --local core.autocrlf false
printf 'safe\n' > policy.txt
git add policy.txt
git commit -qm "Base"
printf 'bad\n' > policy.txt
git commit -qam "Bad policy"
bad=$(git rev-parse HEAD)
printf 'later documentation\n' > README.txt
git add README.txt
git commit -qm "Later docs"
later=$(git rev-parse HEAD)
git revert --no-edit "$bad" > /dev/null
test "$(cat policy.txt)" = safe
test "$(cat README.txt)" = 'later documentation'
git merge-base --is-ancestor "$bad" HEAD
git merge-base --is-ancestor "$later" HEAD
test "$(git rev-list --count HEAD)" -eq 4
printf 'policy restored; later docs kept; bad commit remains in history\n'
```

VERIFIED OUTPUT

```
policy restored; later docs kept; bad commit remains in history
```

> **Why it works:** The inverse changes policy back to safe. The later documentation remains and both earlier commits are still ancestors. History now contains four commits, including the new revert.

CAUTION: Real later commits may depend on the reverted behavior and need additional repair. This fixture deliberately uses an independent later file.

HANDS-ON GIT PRACTICE L07 / L14

TESTED BASH LAB • PASS

L07 Rescue a detached commit with the reflog

Run from a **new empty directory**. Requires Bash and Git; works in Git Bash on Windows.

```
set -eu
# Run in a new, empty disposable directory.
test -z "$(ls -A)"
git init -q -b main
git config --local user.name "Git Learner"
git config --local user.email "learner@example.test"
git config --local commit.gpgsign false
git config --local core.autocrlf false
printf 'base\n' > note.txt
git add note.txt
git commit -qm "Base"
main_tip=$(git rev-parse HEAD)
git switch -q --detach HEAD
printf 'experiment\n' > note.txt
git commit -qam "Detached experiment"
experiment=$(git rev-parse HEAD)
git switch -q main
test "$(git rev-parse HEAD)" = "$main_tip"
lost=$(git rev-parse 'HEAD@{1}')
test "$lost" = "$experiment"
git branch rescue/experiment "$lost"
test "$(git show rescue/experiment:note.txt)" = experiment
test "$(cat note.txt)" = base
printf 'detached experiment rescued; main and its files stayed at base\n'
```

VERIFIED OUTPUT

```
detached experiment rescued; main and its files stayed at base
```

> **Why it works:** In this exact sequence, the most recent previous HEAD position is the detached experiment tip. Creating rescue/experiment makes it reachable under a deliberate name without changing main.

CAUTION: HEAD@{1} is correct only for this known sequence. In an incident, inspect the reflog and verify content instead of assuming that selector always identifies lost work.

HANDS-ON GIT PRACTICE L08 / L14

TESTED BASH LAB • PASS

L08 Backport with cherry-pick, then amend locally

Run from a **new empty directory**. Requires Bash and Git; works in Git Bash on Windows.

```
set -eu
# Run in a new, empty disposable directory.
test -z "$(ls -A)"
git init -q -b main
git config --local user.name "Git Learner"
git config --local user.email "learner@example.test"
git config --local commit.gpgsign false
git config --local core.autocrlf false
printf 'base\n' > README.txt
git add README.txt
git commit -qm "Base"
git switch -qc feature
printf 'validate positive quantity\n' > Order.txt
git add Order.txt
git commit -qm "Validate quantity"
fix=$(git rev-parse HEAD)
git switch -q main
printf 'release docs\n' > README.txt
git commit -qam "Release docs"
git cherry-pick -x "$fix" > /dev/null
picked=$(git rev-parse HEAD)
test "$picked" != "$fix"
test "$(cat Order.txt)" = 'validate positive quantity'
if git merge-base --is-ancestor "$fix" HEAD; then exit 1; fi
printf 'quantity regression test\n' > OrderTest.txt
git add OrderTest.txt
git commit --amend --no-edit -q
test "$(git rev-parse HEAD)" != "$picked"
test -f OrderTest.txt
test "$(git log -1 --format=%s)" = 'Validate quantity'
printf 'backport has new ID; source is not ancestor; amend includes test\n'
```

VERIFIED OUTPUT

```
backport has new ID; source is not ancestor; amend includes test
```

Why it works: The cherry-picked change is applied onto a different parent, so it has a different commit identity without merging feature ancestry. Amend creates another replacement tip while retaining the message and adding the staged test.

CAUTION: The Order files are text fixtures, not Java code. A real backport must compile and pass tests on the release toolchain; shared amendment requires coordination.

HANDS-ON GIT PRACTICE L09 / L14

TESTED BASH LAB • PASS

L09 Fetch, fast-forward pull and a stale push lease

Run from a **new empty directory**. Requires Bash and Git; works in Git Bash on Windows.

```
set -eu
# New empty disposable directory; no network server is used.
test -z "$(ls -A)"
git init --bare -q --initial-branch=main remote.git
git clone -q remote.git alice
for who in alice; do
  git -C "$who" config user.name "Git Learner"
  git -C "$who" config user.email "learner@example.test"
  git -C "$who" config commit.gpgsign false
done
printf 'base\n' > alice/base.txt
git -C alice add base.txt
git -C alice commit -qm "Base"
git -C alice push -qu origin main
git clone -q remote.git bob
git -C bob config user.name "Git Learner"
git -C bob config user.email "learner@example.test"
git -C bob config commit.gpgsign false
base=$(git -C bob rev-parse HEAD)
printf 'A\n' > alice/a.txt
git -C alice add a.txt
git -C alice commit -qm "Alice A"
git -C alice push -q origin main
git -C bob fetch -q origin
test "$(git -C bob rev-parse HEAD)" = "$base"
test "$(git -C bob rev-parse origin/main)" != "$base"
git -C bob pull -q --ff-only
expected=$(git -C bob rev-parse origin/main)
printf 'B\n' > bob/b.txt
git -C bob add b.txt
git -C bob commit -qm "Bob B"
printf 'C\n' > alice/c.txt
git -C alice add c.txt
git -C alice commit -qm "Alice C"
git -C alice push -q origin main
if git -C bob push --force-with-lease=refs/heads/main:"$expected" \
    origin HEAD:refs/heads/main > /dev/null 2>&1; then exit 1; fi
test "$(git --git-dir=remote.git rev-parse main)" = \
    "$(git -C alice rev-parse HEAD)"
printf 'fetch left bob main unchanged; pull advanced it\n'
printf 'stale explicit lease rejected; Alice C stayed on remote\n'
```

VERIFIED OUTPUT

```
fetch left bob main unchanged; pull advanced it
stale explicit lease rejected; Alice C stayed on remote
```

> **Why it works:** Fetch advances Bob's local origin/main, then ff-only pull moves his main. Bob records the reviewed remote tip before Alice publishes C. His later explicit lease no longer matches the remote, so the forced rewrite is rejected.

CAUTION: All pushes target remote.git under this disposable directory. A lease is an update guard, not authorization to rewrite a real shared branch. This exercise intentionally simulates divergence.

HANDS-ON GIT PRACTICE L10 / L14

TESTED BASH LAB • PASS

L10 Stash index, untracked files and a conflicting pop

Run from a **new empty directory**. Requires Bash and Git; works in Git Bash on Windows.

```
set -eu
# Run in a new, empty disposable directory.
test -z "$(ls -A)"
git init -q -b main
git config --local user.name "Git Learner"
git config --local user.email "learner@example.test"
git config --local commit.gpgsign false
git config --local core.autocrlf false
printf 'base\n' > work.txt
git add work.txt
git commit -qm "Base"
printf 'staged\n' > work.txt
git add work.txt
printf 'new\n' > new.txt
git stash push -q -u -m "Saved work"
test ! -e new.txt
test "$(cat work.txt)" = base
git stash apply -q --index
test "$(git show :work.txt)" = staged
test -f new.txt
test "$(git stash list | wc -l)" -eq 1
git stash drop -q
git add new.txt
git commit -qm "Save restored work"
printf 'stash-side\n' > work.txt
git stash push -q -m "Will conflict"
printf 'current-side\n' > work.txt
git commit -qam "New base"
if git stash pop -q > /dev/null 2>&1; then exit 1; fi
test -n "$(git ls-files -u)"
test "$(git stash list | wc -l)" -eq 1
printf 'index and untracked file restored; apply kept stash\n'
printf 'conflicting pop retained stash for resolution\n'
```

VERIFIED OUTPUT

```
index and untracked file restored; apply kept stash
conflicting pop retained stash for resolution
```

> **Why it works:** The first stash includes the untracked new file. Apply --index restores the staged work and leaves the stash entry. Later, a conflicting pop produces unmerged entries and keeps the saved stash rather than dropping it.

CAUTION: The lab ends in a deliberate conflict. In real work, resolve and verify before dropping the stash; do not treat the saved entry as a permanent backup.

HANDS-ON GIT PRACTICE L11 / L14

TESTED BASH LAB • PASS

L11 Parallel worktrees and independent checkouts

Run from a **new empty directory**. Requires Bash and Git; works in Git Bash on Windows.

```
set -eu
# Run in a new, empty disposable directory.
test -z "$(ls -A)"
git init -q -b main
git config --local user.name "Git Learner"
git config --local user.email "learner@example.test"
git config --local commit.gpgsign false
git config --local core.autocrlf false
printf 'base\n' > base.txt
git add base.txt
git commit -qm "Base"
main_tip=$(git rev-parse HEAD)
git worktree add -q -b hotfix ../hotfix main
printf 'fix\n' > ../hotfix/fix.txt
git -C ../hotfix add fix.txt
git -C ../hotfix commit -qm "Hotfix"
test "$(git rev-parse HEAD)" = "$main_tip"
test ! -e fix.txt
test "$(git show hotfix:fix.txt)" = fix
test "$(git -C ../hotfix branch --show-current)" = hotfix
git worktree remove ../hotfix
test ! -d ../hotfix
git show-ref --verify --quiet refs/heads/hotfix
printf 'main unchanged; hotfix committed separately; branch survives removal\n'
```

VERIFIED OUTPUT

```
main unchanged; hotfix committed separately; branch survives removal
```

Why it works: Each checkout has its own working files and checked-out HEAD. Objects and branches are shared, so main can inspect hotfix by reference while its own files remain unchanged. Removing the clean worktree keeps the hotfix branch.

CAUTION: The sibling ../hotfix path must not already contain valuable files. Use a fresh exercise parent directory; worktree add should not be forced over an existing task.

HANDS-ON GIT PRACTICE L12 / L14

TESTED BASH LAB • PASS

L12 Automated bisect finds the first failing commit

Run from a **new empty directory**. Requires Bash and Git; works in Git Bash on Windows.

```
set -eu
# Run in a new, empty disposable directory.
test -z "$(ls -A)"
git init -q -b main
git config --local user.name "Git Learner"
git config --local user.email "learner@example.test"
git config --local commit.gpgsign false
git config --local core.autocrlf false
printf '3\n' > limit.txt
git add limit.txt
git commit -qm "Limit 3"
good=$(git rev-parse HEAD)
printf '5\n' > limit.txt
git commit -qam "Limit 5"
printf '11\n' > limit.txt
git commit -qam "Limit 11: regression"
first_bad=$(git rev-parse HEAD)
printf '12\n' > limit.txt
git commit -qam "Limit 12"
printf '15\n' > limit.txt
git commit -qam "Limit 15"
tip=$(git rev-parse HEAD)
git bisect start "$tip" "$good" > /dev/null
git bisect run sh -c 'test "$(cat limit.txt)" -le 10' > /dev/null 2>&1
test "$(git rev-parse refs/bisect/bad)" = "$first_bad"
git bisect reset > /dev/null 2>&1
test "$(git rev-parse HEAD)" = "$tip"
test "$(git branch --show-current)" = main
printf 'bisect found Limit 11; reset returned to original main tip\n'
```

VERIFIED OUTPUT

```
bisect found Limit 11; reset returned to original main tip
```

> **Why it works:** The test succeeds at limits 3 and 5 and fails from 11 onward. The final refs/bisect/bad identifies the first failing revision; HEAD can remain at the last tested revision. Reset exits bisect and restores the original checkout.

CAUTION: A Java regression wrapper needs a policy for unbuildable revisions, typically skip rather than falsely bad. Flaky or non-monotonic behavior requires further diagnosis.

HANDS-ON GIT PRACTICE L13 / L14

TESTED BASH LAB • PASS

L13 Ignore does not untrack existing content

Run from a **new empty directory**. Requires Bash and Git; works in Git Bash on Windows.

```
set -eu
# Run in a new, empty disposable directory.
test -z "$(ls -A)"
git init -q -b main
git config --local user.name "Git Learner"
git config --local user.email "learner@example.test"
git config --local commit.gpgsign false
git config --local core.autocrlf false
printf 'local setting\n' > local.properties
git add local.properties
git commit -qm "Tracked local file"
printf 'local.properties\ntarget/\n' > .gitignore
git add .gitignore
git commit -qm "Ignore local files"
git ls-files --error-unmatch local.properties > /dev/null
printf 'changed setting\n' > local.properties
test -n "$(git diff -- local.properties)"
git rm -q --cached local.properties
git commit -qm "Stop tracking local setting"
test -f local.properties
test -z "$(git ls-files local.properties)"
git check-ignore -q local.properties
test "$(git show HEAD~2:local.properties)" = 'local setting'
printf 'ignore alone kept tracking; cached removal kept local file\n'
printf 'old content still exists in historical commit\n'
```

VERIFIED OUTPUT

```
ignore alone kept tracking; cached removal kept local file
old content still exists in historical commit
```

> **Why it works:** The ignore entry does not remove a tracked path. Removing it from the index records a deletion while the local copy remains. The older committed content is still reachable in history.

CAUTION: This fixture contains no secret. For an exposed credential, rotation is essential; stopping tracking or changing history does not revoke copied credentials.

HANDS-ON GIT PRACTICE L14 / L14

TESTED BASH LAB • PASS

L14 Annotated and lightweight tag objects

Run from a **new empty directory**. Requires Bash and Git; works in Git Bash on Windows.

```
set -eu
# Run in a new, empty disposable directory.
test -z "$(ls -A)"
git init -q -b main
git config --local user.name "Git Learner"
git config --local user.email "learner@example.test"
git config --local commit.gpgsign false
git config --local core.autocrlf false
printf 'release\n' > README.txt
git add README.txt
git commit -qm "Release source"
release=$(git rev-parse HEAD)
git tag demo-light
git tag -a v1.0.0 -m "Release 1.0.0"
test "$(git cat-file -t demo-light)" = commit
test "$(git cat-file -t v1.0.0)" = tag
test "$(git rev-parse 'v1.0.0^{commit}')" = "$release"
printf 'next version\n' > README.txt
git commit -qam "Next source"
test "$(git rev-parse 'v1.0.0^{commit}')" = "$release"
test "$(git rev-parse HEAD)" != "$release"
printf 'lightweight points to commit; annotated uses a tag object\n'
printf 'new commit did not move release tag\n'
```

VERIFIED OUTPUT

```
lightweight points to commit; annotated uses a tag object
new commit did not move release tag
```

Why it works: The lightweight name resolves directly to a commit. The annotated name first resolves to a tag object; the peel expression selects its commit. Moving the branch with another commit leaves both tag references unchanged.

CAUTION: This verifies ordinary local behavior, not a server-enforced immutability policy or signature trust. Publishing release tags is a separate step.

Quick command decisions

See what changed: `git status --short` and `git diff`

Review staged work: `git diff --staged`

Unstage and keep files: `git restore --staged -- FILE`

Discard unstaged tracked edit: `git restore -- FILE` (destructive)

Undo private tip, keep staged: `git reset --soft TARGET`

Undo private tip, keep unstaged: `git reset --mixed TARGET`

Undo shared change: `git revert COMMIT`

Rescue lost commit: `git reflog` then `git branch rescue/name OID`

See incoming work: `git fetch` then compare tracking refs

Spot the first bad commit: `git bisect`

Core model: `working files → git add → index → git commit → history → git push → remote`. Always name which part a command changes.

14-day interview sprint

Suggested practice sequence; adapt it to your experience.

Days 1–2: Q001–Q015, labs L01–L02. Explain snapshots and staging.

Days 3–4: Q016–Q030, lab L03. Draw branches and merging.

Days 5–6: Q031–Q045, labs L04–L06. Rebase, cherry-pick and undo.

Days 7–8: Q046–Q060, labs L07–L09. Recover and collaborate safely.

Days 9–10: Q061–Q080, labs L10–L11. Tags, worktrees, objects.

Days 11–12: Q081–Q100, labs L12–L14. Debugging, Java CI, security.

Day 13: Q101–Q110. Simulate an incident and defend your fix.

Day 14: Two timed mock interviews: merge conflict and bad deployment.

Mock interview pattern: state the command, show HEAD/index/file changes, explain the risk, then show one verification command.

Official references

These links are taken from the original guide, to verify behavior for the Git version you use. This notebook is an independent study adaptation, not official Git documentation.

https://git-scm.com/docs/git-cat-file

https://git-scm.com/docs/gitrevisions

https://git-scm.com/docs/git-commit

https://git-scm.com/docs/git-diff

https://git-scm.com/docs/git-log

https://git-scm.com/docs/git-config

https://git-scm.com/docs/git-clone

https://git-scm.com/docs/git-switch

https://git-scm.com/docs/git-merge

https://git-scm.com/docs/git-rebase

https://git-scm.com/docs/git-range-diff

https://git-scm.com/docs/git-cherry-pick

https://git-scm.com/docs/git-restore

https://git-scm.com/docs/git-reset

https://git-scm.com/docs/git-revert

https://git-scm.com/docs/git-reflog

https://git-scm.com/docs/git-clean

https://git-scm.com/docs/git-fetch

https://git-scm.com/docs/git-pull/2.51.0

https://git-scm.com/docs/git-push

https://git-scm.com/docs/git-tag

https://git-scm.com/docs/git-stash

https://git-scm.com/docs/git-worktree

https://git-scm.com/docs/gitignore

https://git-scm.com/docs/gitattributes

https://git-scm.com/docs/git-gc

https://git-scm.com/docs/git-bisect

https://git-scm.com/docs/git-blame

https://git-scm.com/docs/git-sparse-checkout

https://git-scm.com/docs/git-submodule

https://git-lfs.com/

https://git-scm.com/docs/githooks

https://git-scm.com/docs/gitcredentials

https://git-scm.com/docs/git-rerere